Github
ACTUALIZADO HASTA LA SECCIÓN 3. Falta: 4, 5, etc.
3 de noviembre de 2026
Versión Extendida

Índice

1 Templates y Testing . . . 1
1.1 Template y archivos . . . 1
1.2 Stress testing (diff) . . . 1
1.3 Stress testing (checker) . . . 1
2 Ideas . . . 1
3 Estructuras básicas . . . 1
3.1 Min queue . . . 1
3.2 Heap actualizable . . . 2
4 Teoría de números . . . 2
4.1 Cribas . . . 2
4.2 Algoritmo extendido de Euclides . . . 2
4.3 Solución de ecuaciones diofánticas lineales . . . 2
4.4 Funciones multiplicativas . . . 2
4.5 Factorización Pollard Rho . . . 3
5 Combinatoria . . . 3
5.1 Binomio . . . 3
5.2 Números de Catalán . . . 3
5.3 Números de Narayana . . . 4
5.4 Desarreglos (Derangements) . . . 4
5.5 Particiones Enteras . . . 4
5.6 Permutaciones: Inversiones y Swaps . . . 4
6 PIE . . . 5
6.1 Principio de inclusión–exclusión . . . 5
6.2 Similares . . . 5
6.3 Divisibilidad y MCM . . . 5
6.4 PIE sobre múltiplos (DP) . . . 5
6.5 Inversión de Möbius . . . 5
7 Tecniques . . . 5
7.1 Conteo por niveles . . . 5
7.2 Agrupación por cociente . . . 6
7.3 Intervalos de cociente constante . . . 6
8 Cálculo . . . 6
8.1 Transformada de Fourier . . . 6
9 Métodos numéricos . . . 6
9.1 Inversa de Matriz con Gauus-Jordan . . . 6
9.2 Sistemas de ecuaciones lineales . . . 6
9.3 Sistemas de ecuaciones modulo 2 . . . 6
10 Sparse table . . . 7
11 Fenwick Tree . . . 7
12 Segment Tree . . . 7
12.1 Actualizaciones puntuales . . . 7
12.2 Actualizaciones sobre rangos . . . 7
12.3 Sparse segment tree . . . 7
13 Sqrt decomposition . . . 7
13.1 Algoritmo de MO . . . 7
14 DSU . . . 8
15 Grafos . . . 8
15.1 Caminos mínimos . . . 8
15.2 Árboles . . . 8
15.3 Matching . . . 10
15.4 Máximo flujo . . . 10
15.5 SCC . . . 12
15.6 2-Sat . . . 12
16 Treap . . . 12
17 Strings . . . 13
17.1 Hashing . . . 13
17.2 KMP . . . 13
17.3 Suffix Automata . . . 13
17.4 Suffix array . . . 13
17.5 Aho-Corasick . . . 14
17.6 Suffix tree . . . 14
18 Geometría . . . 15
18.1 Convex hull . . . 15
19 Teoría de juegos . . . 15
19.1 Green Numbers . . . 15
19.2 Green Hackenbush . . . 15
20 Dinamic Programming . . . 16
20.1 Subset sum optimization. . . . 16
20.2 Subset Sum con bitset . . . 16
20.3 SOS DP . . . 16
20.4 Li-Chao Tree . . . 16
21 Misceláneos . . . 17
21.1 XOR Basis . . . 17
22 Utilidades . . . 17
22.1 Bitsets de tamaño (casi) dinámico . . . 17
22.2 Bitwise ops . . . 17
22.3 Bitwise - builtin . . . 18
22.4 Iterar . . . 18
22.5 Gospers' Hack . . . 18
23 Formulazas . . . 18
23.1 Lógica - Conjuntos - Bitwise . . . 18
23.2 Teoria de numeros . . . 18
23.3 Combinatoria . . . 18
23.3.1 Números de Stirling del primer tipo . . . 18
23.3.2 Números de Stirling del segundo tipo . . . 18
23.3.3 Números de Bernoulli . . . 18
23.3.4 Interpolación de Lagrange . . . 18
23.4 Aritmética . . . 19
23.4.1 Sumas de Gauss . . . 19
23.4.2 Numeros simplices (Sumas de sumas de Gauss) . . . 19
23.4.3 Potencias de base fija (geométrica) . . . 19
23.4.4 Series infinitas . . . 19
23.4.5 Diferencias finitas . . . 19
23.5 Geometría . . . 19
23.6 Desigualdades . . . 19

TEMPLATES Y TESTING

▶ Template y archivos

template.h

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef int64_t ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<vi> vvi;
typedef vector<vll> vvll;
#define fi first
#define se second
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define rep(i,a,b) for(int i = (a); i < (b); ++i)
#define repr(i,a,b) for(int i = (b) - 1; (a) <= i; --i)
const int MOD = 1e9 + 7;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef __int128_t lll;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> ordered_set;
mt19937_64 seed(chrono::steady_clock::now().time_since_epoch()
    .count());
uniform_int_distribution<ll> ale(1, LLONG_MAX);
template<typename C, typename T = typename
    enable_if<!is_same<C, string>::value, typename
    C::value_type>::type>
ostream& operator<<(ostream &os, const C &v);
template<typename A, typename B>
ostream& operator<<(ostream &os, const pair<A, B> &p) {
    return os << '(' << p.fi << ", " << p.se << ')';
}
template<typename C, typename T>
ostream& operator<<(ostream &os, const C &v) {
    string sep;
    for(const T &x : v) os << sep << x, sep = " ";
    return os;
}
constexpr const char *RST="\033[0m", *GREEN="\033[1;32m",
*YELLOW="\033[1;33m",*CYAN="\033[1;36m",*RED="\033[1;31m";
template<typename ...Args>
void logger(const char* col, string vars, Args&&... values){
    cerr << col << '[' << vars << "] =" << RST;
    ((cerr << " [" << values << ']'), ...);
    cerr << '\n';
}
#ifdef LOCAL
    #define DEBUG   if (true)
    #define SALTO   (cerr << '\n')
    #define LOG(...)   logger("", #__VA_ARGS__, __VA_ARGS__)
    #define LOGC(c, ...) logger(c, #__VA_ARGS__, __VA_ARGS__)
    #define COL(c, ...) (cerr << c << __VA_ARGS__ << RST <<
        '\n')
#else
    #define DEBUG   if (false)
    #define SALTO
    #define LOG(...)
    #define LOGC(...)
    #define COL(...)
#endif
int main(){
    #ifdef LOCAL
        ifstream cin("test.in");
    #else
        ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    #endif
}
```

Ejecutar en la terminal de carpeta

```
for f in {a..n}; do cp template.cpp $f.cpp; done
```

▶ Stress testing (diff)

Para stress testing en problemas donde existe solo una respuesta, usamos *stress.sh*. Testeamos *code.cpp* contra *brute.cpp*. *gen.cpp* es el generador de casos. Este ultimo recibe de semilla el valor i.

stress.sh

```
set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
    ./gen $i > input_file
    ./code < input_file > myAnswer
    ./brute < input_file > correctAnswer
    diff -Z myAnswer correctAnswer > /dev/null || break
    echo "Passed test: "  $i
done
echo "WA on the following test:"
cat input_file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer
```

Para extraer la semilla en gen.cpp leemos desde argv

```
int rnd(int a, int b){return a + rand() % (b - a + 1);}
int main(int argc, char* argv[]){
    int seed = atoi(argv[1]);
    srand(seed);
    int n = rnd(1, 5);
}
```

▶ Stress testing (checker)

Usaremos el script similar al anterior *stress.sh*. Aquí el *brute.cpp* es opcional.

```
set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
g++ checker.cpp -o checker
for((i = 1; ; ++i)); do
    ./gen $i > input_file
    ./code < input_file > myAnswer
    ./brute < input_file > correctAnswer
    ./checker > checker_log
    echo "Passed test: "  $i
done
```

Se ejecutara mientras el return value de checker sea 0. Ejemplo de la implementación de un checker

```
bool validAns(ifstream &fin, int n, int* arr){
    fin>>n;
    //aquí validariamos nuestra respuesta, fin sera el input
        stream que le pase
    if(algunacosa) return true
    else return false
}
int main(int argc, char * argv[]){
    //input stream del caso
    ifstream fin("input_file", ifstream::in);
    //input stream de mi respuesta
    ifstream ans("myAnswer", ifstream::in);
    //input stream de respuesta bruta
    ifstream cor("correctAnswer", ifstream::in);
    //aqui fin puede procesar lo que interese del input original
    fin>>n;
    int arr[n];
    for(int x = 0; x < n; x++) cin>>arr[x];
    if(!validAns(ans, n, arr)) {
        cout<<"WA x algun motivo, esto saldrá en cheker_log\n";
        return -1;
    }
    return 0;
}
```

IDEAS

Ideas de Catalán. Ideas que se usan en las demostraciones de que algo se cuenta con Catalán y sirven para conteos similares:

1. *Partir en dos conjuntos y distribuir.* Usar la misma idea de la recurrencia de los números de Catalán: $(0, n-1), (1, n-2), \ldots, (k, n-1-k)$.
2. *Reflejar los caminos malos.* Si queremos contar caminos monótonos que $(0,0)$ y (n,m), por debajo de una diagonal paralela a la que une dichas esquinas, en una cuadrícula de $n \times m$, contemos todos los caminos $\binom{n+m}{n}$ y restemos los caminos que pasan sobre la diagonal. Digamos que la diagonal es $y = x + k$, entonces los caminos malos pasan al menos una vez por $y = x + k + 1$: toma el primer punto $(x', x' + k + 1)$ sobre el que un camino pasa por $y = x + k + 1$ y refléjalo, nota que todos empiezan en $(x', x' + k + 1) - (x' + k + 1, x') = (-k - 1, k + 1)$. Entonces los caminos malos son biyectivos a los caminos que van de $(-k-1, k+1)$ a (n, m).

Optimización. Ideas para optimizar funciones.

1. *Fijar coordenadas secuencialmente.* Hay funciones en R^n que se pueden optimizar fijando la primera coordenada y resolviendo para R^{n-1} y así sucesivamente.

Ad- hoc. Ideas que no son de ninguna categoría en particular.

1. Independecia de eventos (no probabilidad). Idea de poner letras G y T independientes.
2. Encontrar bases/generadores.
   - Base de un espacio vectorial $\mathcal{B}$.
   - Generadores de una sigma-álgebra $\sigma(\mathcal{C})$.
3. En el estado actual, que informacion podemos olvidar y que informacion es necesaria para resolver el problema.

ESTRUCTURAS BÁSICAS

▶ Min queue

min_queue.cpp

```cpp
template<typename T> struct min_stack{
    stack<pair<T, T>> st;
    min_stack(){}
    min_stack(const T &MAXVAL){init(MAXVAL);}
    void init(const T &MAXVAL){st.push({MAXVAL, MAXVAL});}
    void push(const T &v){st.push({v, min(v, st.top().se)});}
    T top(){return st.top().fi;}
    void pop(){if(sz(st) > 1)st.pop();}
    T minV(){return st.top().se;}
    int size(){return sz(st) - 1;}
    bool empty(){return size() == 0;}
};
template<typename T> struct min_queue{
    min_queue(const T &MAXVAL){p_in.init(MAXVAL);
        p_out.init(MAXVAL);}
    void push(const T &v){p_in.push(v);}
    T front(){transfer(); return p_out.top();}
    void pop(){transfer(); p_out.pop();}
    int size(){return sz(p_in) + sz(p_out);}
    T minV() {return min(p_in.minV(), p_out.minV());}
    bool empty(){ return size() == 0;}
    void transfer(){
        if(sz(p_out)) return;
        while(sz(p_in)){p_out.push(p_in.top()); p_in.pop();}
    }
    min_stack<T> p_in, p_out;
};
```

▶ Heap actualizable

updatable_heap.cpp

```cpp
template<class TPriority, class TKey> class UpdatableHeap{
public:
    UpdatableHeap(){
        TPriority a;
        TKey b;
        nodes.clear(); nodes.pb({a, b});
    }
    pair<TPriority, TKey> top(){return nodes[1];}
    void pop(){
        if(sz(nodes) == 1) return;
        TKey k = nodes[1].second;
        swap_nodes(1, sz(nodes) - 1);
        nodes.pop_back();
        position.erase(k);
        heapify(1);
    }
    void insert_or_update(const TPriority &p, const TKey &k){
        int pos;
        if(is_inserted(k)){
            pos = position[k];
            nodes[pos].first += p;
        } else {
            position[k] = pos = sz(nodes);
            nodes.pb({p, k});
        } heapify(pos);
    }
    bool is_inserted(const TKey &k){return position.count(k);}
    int size(){return sz(nodes) - 1;}
    void erase(const TKey &k){
        if(!is_inserted(k)) return;
        int pos = position[k];
        swap_nodes(pos, sz(nodes) - 1);
        nodes.pop_back();
        position.erase(k);
        heapify(pos);
    }
private:
    vector<pair<TPriority, TKey>> nodes;
    map<TKey, int> position;
    void heapify(int pos){
        if(pos >= sz(nodes)) return;
        while(1 < pos && nodes[pos / 2] <= nodes[pos]){
            swap_nodes(pos / 2, pos);
            pos /= 2;
        }
        int l = pos * 2, r = pos * 2 + 1, maxi = pos;
        if(l < sz(nodes) && nodes[l] > nodes[maxi]) maxi = l;
        if(r < sz(nodes) && nodes[r] > nodes[maxi]) maxi = r;
        if(maxi != pos){
            swap_nodes(pos, maxi);
            heapify(maxi);
        }
    }
    void swap_nodes(int a, int b){
        position[nodes[a].se] = b;
        position[nodes[b].se] = a;
        swap(nodes[a], nodes[b]);
    }
};
```

TEORÍA DE NÚMEROS

▶ Cribas

T: $O(n \log\log n)$ | **M:** $O(n)$ |

sieve.cpp

```cpp
void sieve(int n, vi &primes){
    primes.clear(); if(n < 2) return;
    vector<bool> nprime(n + 1);
    nprime[0] = nprime[1] = 1;
    for(ll i = 3; i * i <= n; i += 2) if(!nprime[i])
    for(ll j = i * i; j <= n; j += 2 * i) nprime[j] = 1;
    primes.pb(2);
    for(int i = 3; i <= n; i += 2) if(!nprime[i]) primes.pb(i);
}
```

T: $O((\sqrt{b}+b-a)\log\log\sqrt{b})$ | **M:** $O(\sqrt{b}+b-a)$ | *Encuentra los primos en* $[a,b]$ *con* $b-a \le K \cdot 10^6$

range_sieve.cpp

```cpp
void range_sieve(ll a, ll b, vll &primes){
    a = max(a, 0ll);
    b = max(b, 0ll);
    ll len = b - a + 1;
    vi sqrt_primes;
    sieve(sqrt(b) + 1, sqrt_primes);
    vector<char> nprime(len);
    primes.clear();
    for(ll p : sqrt_primes){
        ll ini = p * max(p, (a + p - 1) / p);
        for(ll m = ini; m <= b; m += p) nprime[m - a] = 1;
    }
    for(ll i = 0; i < len; ++i)
        if(!(nprime[i] || i + a < 2)) primes.pb(i + a);
}
```

T: $O((n+\sqrt{n})\log\log\sqrt{n})$ | **M:** $O(\sqrt{n}+S)$ | *Cuenta primos menores o iguales a* n.

segmented_sieve.cpp

```cpp
int count_primes(int n){
    if(n < 2) return 0;
    const int S = sqrt(n);
    vi sqrt_primes;
    sieve(sqrt(n) + 1, sqrt_primes);
    int ans = 0;
    vector<char> nprime(S + 1);
    for(int ini = 0; ini <= n; ini += S){
        fill(all(nprime), 0);
        for(int p : sqrt_primes){
            ll m = 1ll * p * max(p, (ini + p - 1) / p) - ini;
            for(; m <= S; m += p) nprime[m] = 1;
        }
        for(int i = 0; i < S && i + ini <= n; ++i)
            if(!nprime[i] && 1 < i + ini) ans++;
    } return ans;
}
```

T: $O(n)$ | **M:** $O(n)$ | *Para calcular funciones multiplicativas difíciles de adaptar en la criba de Eratóstenes es mejor la criba lineal.*

linear_sieve.cpp

```cpp
void linear_sieve(int n, vi &primes){
    primes.clear();
    if(n < 2) return;
    vi lp(n + 1);
    for(ll i = 2; i <= n; ++i){
        if(!lp[i]) primes.pb(lp[i] = i);
        for(int j = 0; i * primes[j] <= n; ++j){
            lp[i * primes[j]] = primes[j];
            if(primes[j] == lp[i]) break;
        }
    }
}
```

▶ Algoritmo extendido de Euclides

T: $O(\log \min(a,b))$ | **M:** $O(1)$ | *Encuentra una solución de la ecuación* $ax+by=\gcd(a,b)$.

extended_gcd.cpp

```cpp
ll ext_gcd(ll a, ll b, ll &x, ll &y){
    if(!b) return x = 1, y = 0, a;
    ll g = ext_gcd(b, a % b, y, x);
    return y -= a / b * x, g;
}
```

▶ Solución de ecuaciones diofánticas lineales

T: $O(\log \min(a,b))$ | **M:** $O(1)$ | *Encuentra o cuenta soluciones de la ecuación* $ax+by=c$.

diophantine_equations.cpp

```cpp
bool find_sol(ll a, ll b, ll c, ll &x, ll &y, ll &g){
    g = ext_gcd(abs(a), abs(b), x, y);
    if(c % g) return false;
    x *= c / g;
    y *= c / g;
    if(a < 0) x = -x;
    if(b < 0) y = -y;
    return true;
}
void change_sol(ll &x, ll &y, ll a, ll b, ll cnt, ll g = 1){
    x += cnt * b / g;
    y -= cnt * a / g;
}
ll cntsol(ll a, ll b, ll c, ll minx, ll maxx, ll miny, ll maxy){
    ll x, y, g;
    if(!find_sol(a, b, c, x, y, g)) return 0;
    a /= g; b /= g;
    int sign_a = a > 0 ? +1 : -1; int sign_b = b > 0 ? +1 : -1;
    change_sol(x, y, a, b, (minx - x) / b);
    if(x < minx) change_sol(x, y, a, b, sign_b);
    if(x > maxx) return 0;
    ll lx1 = x;
    change_sol(x, y, a, b, (maxx - x) / b);
    if(x > maxx) change_sol(x, y, a, b, -sign_b);
    ll rx1 = x;
    change_sol(x, y, a, b, -(miny - y) / a);
    if(y < miny) change_sol(x, y, a, b, -sign_a);
    if(y > maxy) return 0;
    ll lx2 = x;
    change_sol(x, y, a, b, -(maxy - y) / a);
    if(y > maxy) change_sol(x, y, a, b, sign_a);
    ll rx2 = x;
    if(lx2 > rx2) swap(lx2, rx2);
    ll lx = max(lx1, lx2); ll rx = min(rx1, rx2);
    if(lx > rx) return 0;
    return (rx - lx) / abs(b) + 1;
}
```

▶ Funciones multiplicativas

T: $O(n \log\log n)$ | **M:** $O(n)$ | *Calcula* $\phi(i)$ *para todo* $i \in [1,n]$.

phi_euler.cpp

```cpp
vi calc_phi(int n){
    vi phi(n + 1);
    for(int i = 0; i <= n; ++i) phi[i] = i & 1 ? i : i / 2;
    for(int i = 3; i <= n; i += 2) if(phi[i] == i)
    for(int j = i; j <= n; j += i) phi[j] -= phi[j] / i;
    return phi;
}
```

Para p primo y $a,b,n,k \in \mathbb{Z}^+$ se cumple

$$\phi(p^k) = p^{k-1}(p-1),$$

$$\phi(ab) = \phi(a)\phi(b)\frac{\gcd(a,b)}{\phi(\gcd(a,b))},$$

$$\phi(n) = n\prod_{p|n}\left(1-\frac{1}{p}\right),$$

$$n = \sum_{d|n}\phi(d).$$

Si $G_d = \{x : 1 \le x \le n, \gcd(x,n) = d\}$ entonces $|G_d| = \phi(\frac{n}{d})$. Si $\gcd(x,m) = 1$ entonces $x^{\phi(m)} \equiv 1 \pmod m$. Si $\gcd(x,m) \neq 1$ entonces $x^n \equiv x^{\phi(m)+[n \bmod \phi(m)]} \pmod m$.

Función σ_x. Si p es primo y $a > 0$ entonces

$$\sigma_x(p^a) = \sum_{i=0}^{a} p^{ix} = \frac{p^{x(a+1)}-1}{p^x-1}.$$

Si $\gcd(n,m) = 1$ entonces $\sigma_x(nm) = \sigma_x(n)\sigma_x(m)$. En particular, $\sigma_0(n)$ es la cantidad de divisores de n y $\sigma_1(n)$ es la suma de los

divisores de n. **NOTA:** varios problemas que involucran suma de dividores, suma de suma de divisores, suma de la cantidad de divisores de los divisores, etc, pueden resolverse con argumentos combinatorios para cada primo y luego multiplicando los resultados. Por ejemplo, si $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ entonces

$$\sum_{d|n} d = \prod_{i=1}^{k} \sigma_1\left(p_i^{a_i}\right) = \prod_{i=1}^{k} \frac{p_i^{a_i+1}-1}{p_i-1},$$

$$\sum_{d|n} \sigma_0(d) = \prod_{i=1}^{k} \sum_{j=0}^{a_i} (j+1) = \prod_{i=1}^{k} \frac{(a_i+1)(a_i+2)}{2}.$$

T: $O(n)$ | **M:** $O(n)$ | *Criba lineal para calcular σ_0 o σ_1.*

sigma_fnc.cpp

```cpp
void linear_sigma(int n, vi &lp, vll &sigma1){
    vi primes;
    lp.resize(n + 1, 0);
    sigma1.resize(n + 1, 0);
    vll sum_p(n + 1, 0);
    sigma1[1] = 1;
    for(ll i = 2; i <= n; ++i){
        if(lp[i] == 0){
            lp[i] = i;
            primes.pb(i);
            sigma1[i] = i + 1;
            sum_p[i] = i + 1;
        }
        for(int p : primes){
            if(p > lp[i] || i * p > n) break;
            lp[i * p] = p;
            if(lp[i] == p){
                sum_p[i*p] = sum_p[i] * p + 1;
                sigma1[i*p] = sigma1[i] / sum_p[i] * sum_p[i*p];
                break;
            } else {
                sum_p[i * p] = p + 1;
                sigma1[i * p] = sigma1[i] * (p + 1);
            }
        }
    }
}
```

Función de Möbius. Para un entero $n \geq 1$, $\mu(n)$ se define como:

$$\mu(n) = \begin{cases} 1, & \text{si } n = 1, \\ 0, & \text{si } n \text{ tiene un factor primo al cuadrado,} \\ (-1)^k, & \text{si } n \text{ es el producto de } k \text{ primos distintos.} \end{cases}$$

T: $O(n)$ | **M:** $O(n)$ | *Criba lineal para calcular la función de Möbius.*

mobius.cpp

```cpp
void linear_mobius(int n, vi &primes, vi &mu) {
    vi lp(n + 1);
    mu.assign(n + 1, 0);
    mu[1] = 1;
    for(ll i = 2; i <= n; ++i){
        if(!lp[i]){
            primes.pb(lp[i] = i);
            mu[i] = -1;
        }
        for(int j = 0; i * primes[j] <= n; ++j){
            lp[i * primes[j]] = primes[j];
            if(primes[j] == lp[i]){
                mu[i * primes[j]] = 0;
                break;
            } else mu[i * primes[j]] = -mu[i];
        }
    }
}
```

► Factorización Pollard Rho

Complejidad: $O(\sqrt[4]{n})$. Inicializar con PollardRho::init() y para factorizar un número PollardRho::factorize(n).

```cpp
//COPIADO Y PEGADOPORCUESTIONES DE TIEMPO
namespace PollardRho {
    mt19937 rnd(chrono::steady_clock::now()
    .time_since_epoch().count());
    const int P = 1e6 + 9;
    ll seq[P];
    int primes[P], spf[P];
    inline ll add_mod(ll x, ll y, ll m){
        return (x += y) < m ? x : x - m; }
    inline ll mul_mod(ll x, ll y, ll m) {
        ll res = __int128(x) * y % m;
        return res;
        // ll res = x * y - (ll)((long double)x * y / m + 0.5) *
        ↪  m;
        // return res < 0 ? res + m : res;
    }
    inline ll pow_mod(ll x, ll n, ll m) {
        ll res = 1 % m;
        for (; n; n >>= 1) {
            if (n & 1) res = mul_mod(res, x, m);
            x = mul_mod(x, x, m);
        }
        return res;
    }
    // O(it * (logn)^3), it = number of rounds performed
    inline bool miller_rabin(ll n) {
        if (n<=2 || (n & 1 ^ 1)) return (n==2);
        if (n < P) return spf[n] == n;
        ll c, d, s = 0, r = n - 1;
        for (; !(r & 1); r >>= 1, s++) {}
        // each iteration is a round
        for(int i=0; primes[i]<n && primes[i]<32; i++){
            c = pow_mod(primes[i], r, n);
            for(int j = 0; j < s; j++){
                d = mul_mod(c, c, n);
                if(d==1&&c!=1&&c!=n-1) return 0;
                c = d;
            } if (c!=1) return 0;
        } return 1;
    }
    void init() {
        int cnt = 0;
        for(int i = 2; i < P; i++){
            if(!spf[i]) primes[cnt++] = spf[i]=i;
            for(int j=0,k;(k=i*primes[j])<P;j++){
                spf[k] = primes[j];
                if (spf[i] == spf[k]) break;
            }
        }
    }
    // returns O(n^(1/4))
    ll pollard_rho(ll n) {
        while(1){
            ll x=rnd()%n,y=x,c=rnd()%n,u=1,v,t=0;
            ll *px = seq, *py = seq;
            while(1){
                *py++ = y = add_mod(mul_mod(y, y, n), c, n);
                *py++ = y = add_mod(mul_mod(y, y, n), c, n);
                if((x = *px++) == y) break;
                v = u;
                u = mul_mod(u, abs(y - x), n);
                if(!u) return __gcd(v, n);
                if(++t == 32){
                    t = 0;
                    if((u = __gcd(u,n))>1 && u<n)
                        return u;
                }
            }
            if(t && (u = __gcd(u, n)) > 1 && u<n)
                return u;
        }
    }
    vector<ll> factorize(ll n) {
        if(n == 1) return vector <ll>();
        if(miller_rabin(n)) return vector<ll>{n};
        vector<ll> v, w;
        while(n > 1 && n < P){
            v.pb(spf[n]); n /= spf[n];
        }
        if(n >= P){
            ll x = pollard_rho(n);
            v = factorize(x);
            w = factorize(n / x);
            v.insert(v.end(), all(w));
        } return v;
    }
}
```

COMBINATORIA

► Binomio

Teorema del binomio generalizado. Para $x \in \mathbb{R}$ y $a, b \in \mathbb{C}$ se cumple

$$(a+b)^x = \sum_{k=0}^{\infty} \binom{x}{k} a^x b^{x-k},$$

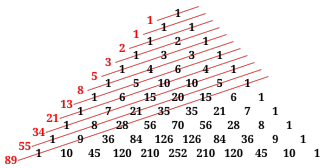
Palo de Hockey. Para n y r $(n \geq r)$, se cumple:

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}, \quad \text{equivalente} \quad \sum_{i=0}^{n} \binom{r+i}{r} = \binom{r+n+1}{r+1}$$

Es fibonacci.

$$F_{n+1} = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n-k}{k},$$

donde F_{n+1} es el $(n+1)$-ésimo número de Fibonacci.



► Números de Catalán

Se puede calcular con $C_0 = C_1 = 1$,

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k} = \frac{1}{n+1}\binom{2n}{n} = \frac{4n+2}{n+2} C_{n-1}, \; n \geq 2.$$

El n-ésimo número de Catalán C_n cuenta

- La cantidad de secuencias de paréntesis balanceadas de longitud $2n$.
- La cantidad de maneras distintas de agrupar $n+1$ factores con paréntesis.
- La cantidad de triangulaciones de un polígono convexo de $n+2$ lados.
- La cantidad de maneras de unir $2n$ puntos en una circunferencia con cuerdas sin que ningún par se corte.
- La cantidad de árboles binarios completos con n nodos *internos* no isomorfos. Los nodos *internos* son aquellos con dos hijos.
- La cantidad de árboles binarios enraizados completos no isomorfos con $n+1$ hojas.
- La cantidad de árboles enraizados planos no isomorfos con $n+1$ nodos.

- La cantidad de árboles binarios no isomorfos con exactamente n nodos.
- La cantidad de caminos monótonos en un tablero de $n \times n$ que van de $(0,0)$ a (n,n) sin que cruce la diagonal que une $(0,0)$ con (n,n).
- La cantidad de permutaciones de tamaño n que son ordenables con una pila (mientras top() $\leq x$, pop(). Luego push(x). Al final haz pop() de los elementos restantes de la pila). Equivalentemente la cantidad de permutaciones que no contienen el patrón 231: no existen índices $i < j < k$ tales que $a_k < a_i < a_j$.
- La cantidad de permutaciones de tamaño n que no contienen el patrón 123: no existen índices $i < j < k$ tales que $a_i < a_j < a_k$.
- La cantidad de particiones no cruzadas de un conjunto de tamaño n.
- La cantidad de maneras de cubrir una escalera con n escalones, con la altura del i-ésimo escalón siendo i, mediante n rectángulos.
- La cantidad de maneras de unir n cuadros de 1×1 tales que cada cuadro tenga a otro cuadro adyacente a sus lados y, cada columna de cuadros tenga una altura absoluta mayor o igual a la altura absoluta a la columna previa. Cada uno de estos polígonos tiene un perímetro de $2n+2$.

Triángulo de Catalán. Sean $n, k \in \mathbb{Z}_{\geq 0}$, definamos

$$C_{n,k} = \begin{cases} 0, & n < k \text{ o } n,k < 0, \\ 1, & n = k = 0, \\ C_{n,k-1} + C_{n-1,k}, & k \leq n. \end{cases}$$

Entonces $C_{n,n} = C_n = \sum_{k=1}^{n-1} C_{n-1,k}$. El número $C_{n,k}$ se puede interpretar como la cantidad de caminos desde (n,k) del Triángulo de Catalán hasta $(0,0)$. Por lo que tiene sentido que $C_{n,n} = C_n$, dado que $C_{n,n}$ es la cantidad de caminos monótonos desde (n,n) a $(0,0)$ en un tablero de $n \times n$ que no cruzan la diagonal que une $(0,0)$ y (n,n).

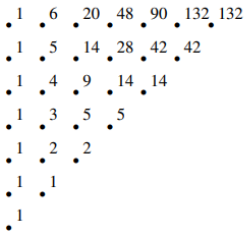


Figura 5.1: Triángulo de Catalán hasta $n = 6$.

► Números de Narayana

Sean $n, k \in \mathbb{Z}^+$ con $k \leq n$, los números de Narayana se definen como

$$N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}.$$

Se cumple que

$$N(n,k) = N(n, n-k+1),$$

$$\sum_{k=1}^{n} N(n,k) = C_n,$$

donde C_n es el n-ésimo número de Catalán. El número $N(n,k)$ cuenta

- La cantidad de secuencias de paréntesis balanceadas con $2n$ paréntesis y k distintos anidamientos, es decir, k ocurrencias de la subcadena ().
- La cantidad de caminos distintos desde $(0,0)$ a $(2n,0)$ dando pasos hacia arriba o abajo y siempre avanzando una unidad en cada paso (diagonales), de manera que haya exactamente k picos. Equivalentemente a la cantidad de maneras de ordenar n 1's y n (-1)'s en una secuencia a_i tales que si S_i es la suma parcial de a_i, entonces $\{S_i\}$ tiene exactamente k máximos locales.

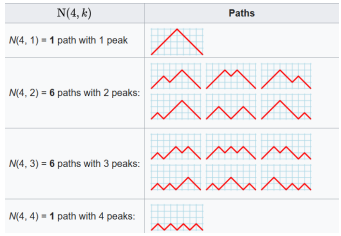


Figura 5.2: Caminos de $(0,0)$ a $(8,0)$ que cuenta $N(4,k)$.

- La cantidad de árboles enraizados con $n+1$ nodos y k hojas.
- La cantidad de particiones no cruzadas de un conjunto de tamaño n en exactamente k subconjuntos.

► Desarreglos (Derangements)

Considera los conjuntos $\{A_k\}_{k=1}^{k=n}$ donde A_k es la cantidad de permutaciones con el punto fijo $p_k = k$. La cantidad de permutaciones con al menos un punto fijo es

$$P := \left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\varnothing \neq J \subseteq [n]} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

$$= \sum_{\varnothing \neq J \subseteq [n]} (-1)^{|J|-1}(n-|J|)! = \sum_{i=1}^{n} (-1)^{i-1}\binom{n}{i}(n-i)!,$$

entonces la cantidad de desarreglos es $n! - P$.
Denotamos por $!n = D_n = D_{n,0}$: Cantidad de permutaciones de n elementos sin ningún punto fijo ($\pi(i) \neq i, \forall i$).

Fórmulas y Recurrencias:

- $D_n = (n-1)(D_{n-1} + D_{n-2})$ con $D_0 = 1, D_1 = 0$.
- $D_n = nD_{n-1} + (-1)^n$ para $n \geq 1$.
- (PIE): $D_n = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!} = n!\left(1 - \frac{1}{1!} + \frac{1}{2!} - \cdots + \frac{(-1)^n}{n!}\right)$.
- $D_n = \left\lfloor \frac{n!+1}{e} \right\rfloor = \text{round}\left(\frac{n!}{e}\right)$ para $n \geq 1$.
- $\lim_{n\to\infty} \frac{D_n}{n!} = \frac{1}{e} \approx 0.367879$.
- **Exactamente k puntos fijos:** $D_{n,k} = \binom{n}{k} D_{n-k}$.
- $\sum_{k=0}^{n} \binom{n}{k} D_k = n!$.

Valores iniciales ($n = 0 \ldots 6$)**:** $1, 0, 1, 2, 9, 44, 265$.
Objetos que cuenta D_n:

- Repartos de n objetos a n destinatarios donde nadie recibe el correcto (ej. cartas en sobres).
- Matrices de permutación de $n \times n$ con ceros en la diagonal principal.
- Grafos dirigidos 1-regulares de n vértices sin autobucles (self-loops).

► Particiones Enteras

T: $O(\frac{13\sqrt{n}}{n})$ | **M:** $O(n)$ | *Genera todas las particiones de un número n como suma de enteros positivos en orden no creciente.*

```
1 void generar(int n, int maximo, vi &actual){
2     if(n == 0){// Aqui trabajar con actual
3         return; }
4     for(int i = min(n, maximo); i >= 1; --i){
5         actual.pb(i);
6         generar(n - i, i, actual);
7         actual.pop_back();
8     }
9 }
```

La función de partición $p(n)$, que cuenta el número de formas distintas de escribir n como suma de enteros positivos sin importar el orden,

n	1	2	3	4	5	6	7	8
$p(n)$	1	2	3	5	7	11	15	22
n	9	10	11	12	13	14	15	16
$p(n)$	30	42	56	77	101	135	176	231

Tabla 1: Particiones enteras

Se tiene la siguiente aproximación asintótica para valores grandes de n, conocida como la **fórmula de Hardy-Ramanujan**:

$$p(n) \sim \frac{1}{4n\sqrt{3}} \exp\left(\pi\sqrt{\frac{2n}{3}}\right)$$

► Permutaciones: Inversiones y Swaps

Sea P una permutación de tamaño n sobre $\{0, \ldots, n-1\}$:

- **Inversión:** Par de índices (i,j) tal que $i < j$ y $P[i] > P[j]$.
- **Swaps Adyacentes mín.:** Iguales a la cantidad de **inversiones**.
- **Swaps Generales mín.:** $n - c(P)$, donde $c(P)$ es el número de ciclos en $i \to P[i]$.
- **Paridad:** #Inversiones $\equiv n - c(P) \pmod 2$.

Distancia entre permutaciones $A \to B$: Para hallar el costo de transformar A en B, se mapean los elementos de A a las posiciones que deben ocupar en B:

1. Construir `pos` tal que `pos`$[B[i]] = i$.
2. Generar $P[i] =$ `pos`$[A[i]]$.
3. La distancia en swaps adyacentes/generales es el número de inversiones/$n - c(P)$ de P.

Peores casos y distancias máximas ($A \to B$):

- **Inversiones (Swaps ady.):** $B = \text{reverse}(A)$, Costo $= n(n-1)/2$.
- **Swaps Generales:** $B = \text{shift_circular}(A)$ (1 solo ciclo), Costo $= n-1$.

T: $O(n \log n)$ | **M:** $O(n)$ |

```cpp
long long merge_inv(vector<int>& a, int l, int r, vector<int>& tmp) {
    if (r - l <= 1) return 0;
    int m = l + (r - l) / 2;
    long long inv = merge_inv(a, l, m, tmp) + merge_inv(a, m, r, tmp);
    int i = l, j = m, k = l;
    while (i < m && j < r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else { tmp[k++] = a[j++]; inv += (m - i); }
    }
    while (i < m) tmp[k++] = a[i++];
    while (j < r) tmp[k++] = a[j++];
    for (int t = l; t < r; t++) a[t] = tmp[t];
    return inv;
}
long long contar_inversiones(vector<int> a) {
    vector<int> tmp(a.size());
    return merge_inv(a, 0, a.size(), tmp);
}
```

PIE

▸ Principio de inclusión–exclusión

Sean $A_1, \ldots, A_m \subseteq U$ los objetos que cumplen cada propiedad. Para contar los que cumplen **alguna**:

$$\left|\bigcup_{i=1}^{m} A_i\right| = \sum_{\emptyset \neq S \subseteq [m]} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|.$$

Impar suma, par resta. Para contar los que no cumplen ninguna, restar la unión a $|U|$.

Si la intersección depende sólo de $|S|$, agrupar con $\binom{m}{|S|}$.

T: $O(2^m T)$ | **M:** $O(1)$ | *T: costo de una intersección*

```cpp
// inter(s): cardinalidad; 0 <= m < 63.
template<class F>
__int128 PIE(int m, F inter){
  __int128 ans = 0;
  for(ll s=1; s<(1LL<<m); ++s){
    __int128 x = inter(s);
    ans += (__builtin_popcountll(s)&1) ? x : -x;
  } return ans;
}
```

▸ Similares

Cantidad de elementos en exactamente r conjuntos. Sean $A_1, \ldots, A_n$ conjuntos y $0 \le r \le n$, la cantidad de elementos en exactamente r conjuntos A_i es

$$\sum_{m=r}^{n} (-1)^{m-r} \binom{m}{r} \sum_{\substack{J \subseteq [n] \\ |J|=m}} \left|\bigcap_{j \in J} A_j\right|$$

Estrellas y barras con cotas. Para $d \ge 1$, $s, b \ge 0$, soluciones de $\sum_{i=1}^{d} x_i = s$, $0 \le x_i \le b$:

$$\sum_{j=0}^{d} (-1)^j \binom{d}{j} \binom{s - j(b+1) + d - 1}{d-1}.$$

Con cotas b_i distintas, sumar por $S \subseteq [d]$ el término $(-1)^{|S|} \binom{s - \sum_{i \in S} (b_i+1) + d - 1}{d-1}$.

▸ Divisibilidad y MCM

Para $A_i = \{x \in [L, R] : a_i \mid x\}$, $a_i > 0$, la intersección exige ser múltiplo de $q = \text{mcm}(a_i : i \in S)$:

$$|A_S| = \lfloor R/q \rfloor - \lfloor (L-1)/q \rfloor.$$

```cpp
auto inter = [&](ll s)->ll{
  ll q=1;
  for(int i=0; i<(int)a.size(); ++i)
    if(s & (1LL<<i)){
      ll t=q/gcd(q,a[i]);
      if(t>R/a[i]) return 0; // ANTES de multiplicar
      q=t*a[i];
    }
  return R/q-(L-1)/q;
};
```

Si tenemos comprimos dos a dos MCM = es solo el producto. Quitar duplicados, $a_i > R$ y a_j si otro $a_i \mid a_j$. Costo $O(m 2^m \log R)$; con primos distintos, $O(m 2^m)$.

Usos: no múltiplos: $R - L + 1 -$ PIE. Coprimos con q: excluir sus primos distinto. Universo pequeño: marcar múltiplos en $O(R + \sum_i R/a_i)$.

▸ PIE sobre múltiplos (DP)

Queremos contar cuantos elementos (parejas, ternas, conjuntos, ...) $x \in S$ cumplen que $\gcd(x) = 1$. Definimos $g(d)$ = cantidad de elementos con $d \mid \gcd$, y $f(d)$ = cantidad con $\gcd = d$.

$$g(d) = \sum_{k=1}^{\lfloor V/d \rfloor} f(kd), \qquad f(d) = g(d) - \sum_{k=2}^{\lfloor V/d \rfloor} f(kd).$$

T: $O(V \log V)$ | **M:** $O(V)$ |

```cpp
vector<ll> f(V+1);
for(int d=V; d>=1; --d){
  f[d]=g[d]; // g(d): CAMBIAR segun caso
  for(int k=2*d; k<=V; k+=d)
    f[d]-=f[k]; // modulo: (f[d]-f[k]+MOD)%MOD
}
```

Para $c =$ `multiplos_de[d]`, cambiar sólo $g(d)$:

Pares de índices $i < j$	$\binom{c}{2}$
Subconjuntos de tamaño k	$\binom{c}{k}$
Subconjuntos no vacíos	$2^c - 1$
Tuplas ordenadas, repetición, tamaño k	c^k

Como calcular cuantos multiplos de d hay en el conjunto :

T: $O(V \log V)$ | **M:** $O(V)$ |

```cpp
for(int x:a) ++frec[x]; // a ya contiene la entrada
for(int d=1; d<=V; ++d)
  for(int k=d; k<=V; k+=d)
    mult[d]+=frec[k];
```

▸ Inversión de Möbius

Usar μ y su criba de Teoría de números. Identidad: $\sum_{d \mid n} \mu(d) = [n = 1]$; codifica PIE sobre primos distintos.

Divisores: Se usa cuando una funcion conocida $g(n)$ acumula valores de una funcion oculta $f(n)$ sobre todos los divisores de n.

$$g(n) = \sum_{d \mid n} f(d) \implies f(n) = \sum_{d \mid n} \mu(d) g(n/d).$$

Múltiplos (dominio $[v]$): En este caso $g(d)$ acumula los valores de $f(k)$ sobre todos los multiplos de d hasta cierto limite superior V

$$g(d) = \sum_{k=1}^{\lfloor V/d \rfloor} f(kd) \implies f(d) = \sum_{k=1}^{\lfloor V/d \rfloor} \mu(k) g(kd).$$

En particular $f(1) = \sum_{d=1}^{V} \mu(d) g(d)$.

Nota: DP para todas las clases, Möbius para $f(1)$ en $O(V)$ si μ está precalculado y $g(d)$ cuesta $O(1)$.

Versión Extendida:

Contar enteros libres de cuadrados. Queremos contar los $x \in [1, N]$ que no son divisibles por ningún cuadrado mayor que 1. Basta excluir los eventos $p^2 \mid x$ para cada primo p. Una intersección exige $d^2 \mid x$, donde d es producto de los primos elegidos; $\mu(d)$ incorpora los signos de PIE:

$$Q(N) = \sum_{d=1}^{\lfloor \sqrt{N} \rfloor} \mu(d) \left\lfloor \frac{N}{d^2} \right\rfloor.$$

Incluye el 1. Precalcular μ hasta $\lfloor \sqrt{N_{\text{máx}}} \rfloor$; cada consulta cuesta $O(\sqrt{N})$. En $[L, R]$: $Q(R) - Q(L-1)$, con $Q(0) = 0$.

Contar números que no son potencias perfectas. Queremos contar los $x \in [2, N]$ que no pueden escribirse como a^b con enteros $a, b \ge 2$. Si $x = \prod_i p_i^{e_i}$, definir $h(x) = \gcd(e_1, e_2, \ldots)$. Entonces x es potencia d-ésima si y sólo si $d \mid h(x)$. Por tanto,

$$g(d) = \#\{x \in [2, N] : d \mid h(x)\} = \lfloor N^{1/d} \rfloor - 1.$$

El -1 excluye la base 1. Invertir sobre múltiplos para obtener $f(d) = \#\{x : h(x) = d\}$; responder $f(1)$. Basta $1 \le d \le \lfloor \log_2 N \rfloor$, pues la menor base es 2. Calcular raíces enteras exactas: búsqueda binaria comparando a^d con N sin overflow; ajustar también las aproximaciones de `sqrt`.

Cuidados de implementación. n es la cantidad de entradas, m la cantidad de eventos y V, L valores máximos.

TECNIQUES

- Cambiar numeros por sumas $\text{mín}(A, x) = \sum_{i=1}^{A} \mathbf{1}_{\{i \le x\}}$, $a = \sum_{i=1}^{a} 1$.
- Fubinni para sumas.
- Contribuction tecnique.

Sean $0 \le a_i \le M$, $M \ge 1$ y $K \ge 0$ enteros. Definimos

$$\text{pf}[y] = \#\{i : a_i \le y\}, \qquad \text{sf}[y] = \#\{i : a_i \ge y\},$$

$$\text{pf}[y] = \sum_{i=1}^{N} \mathbf{1}_{\{a_i \le y\}} \quad \text{y} \quad \text{sf}[y] = \sum_{i=1}^{N} \mathbf{1}_{\{a_i \ge y\}}$$

con $\text{pf}[-1] = 0$ y $\text{sf}[y] = 0$ si $y > M$. Precalcular ambas tablas: $O(N + M)$ tiempo, $O(M)$ memoria.

▸ Conteo por niveles

Cada valor aporta uno por cada umbral $j \cdot x \le a_i$:

$$\sum_{i}^{N} \text{mín}\left(\left\lfloor \frac{a_i}{x} \right\rfloor, K\right) = \sum_{j=1}^{\text{mín}(K, \lfloor M/x \rfloor)} \text{sf}[j \cdot x].$$

T: $O(M \log M)$ | **M:** $O(M)$ | *Todos los $x = 1, \ldots, M$.*

```cpp
for (int x = 1; x <= M; ++x) {
    long long sum = 0;
    for (int j = 1; j <= min(K, M / x); ++j)
        sum += sf[j * x];
    ans[x] = sum;
}
```

Generalización: cada nivel aporta $f(j) - f(j-1)$.

$$\sum_i f\left(\left\lfloor \frac{a_i}{x} \right\rfloor\right) = Nf(0) + \sum_{j=1}^{\lfloor M/x \rfloor} \big(f(j) - f(j-1)\big)\mathrm{sf}[j \cdot x].$$

También:

$$\sum_i \left\lceil \frac{a_i}{x} \right\rceil = \sum_{j=0}^{\lfloor (M-1)/x \rfloor} \mathrm{sf}[j \cdot x + 1],$$

▸ Agrupación por cociente

$\lfloor a_i/x \rfloor = c$ exactamente cuando $a_i \in [cx, (c+1)x - 1]$. Así,

$$\sum_i f\left(\left\lfloor \frac{a_i}{x} \right\rfloor\right) = \sum_{c=0}^{\lfloor M/x \rfloor} f(c)\Big(\mathrm{pf}[\min(M, (c+1)x - 1)] - \mathrm{pf}[cx - 1]\Big).$$

T: $O(M \log M)$ | **M:** $O(M)$ | *Todos los x; evaluar f cuesta $O(1)$.*

```cpp
for (int x = 1; x <= M; ++x) {
    long long sum = 0;
    for (int c = 0; c <= M / x; ++c) {
        int L = c * x;
        int R = min(1LL * M, (c + 1LL) * x - 1);
        int cnt = pf[R] - (L ? pf[L - 1] : 0);
        sum += 1LL * f(c) * cnt;
    }
    ans[x] = sum;
}
```

▸ Intervalos de cociente constante

Para un numerador fijo $A \geq 1$, si $q = \lfloor A/l \rfloor$, entonces $\lfloor A/x \rfloor = q$ para todo $x \in [l, r]$, donde $r = \lfloor A/q \rfloor$. Hay $O(\sqrt{A})$ intervalos.
T: $O(\sqrt{A})$ | **M:** $O(1)$ | $\sum_{x=1}^{A} f(\lfloor A/x \rfloor)$; *$f$ en $O(1)$.*

```cpp
long long sum = 0;
for (long long l = 1, r; l <= A; l = r + 1) {
    long long q = A / l;
    r = A / q;
    sum += (r - l + 1) * f(q);
}
```

CÁLCULO

▸ Transformada de Fourier

FFT. Complejidad: Tiempo $O(n \log n)$ - Memoria extra $O(n \log n)$, donde n es el grado del polinomio P.

```cpp
using comp = complex<double>;
const double PI = acos(-1);
vector<comp> FFT(vector<comp> &P, bool inversa){
    int n = sz(P);
    if(n == 1) return P;
    vector<comp> Pe, Po;
    for(int i = 0; i < n; ++i)
        if(i % 2) Po.pb(P[i]);
        else Pe.pb(P[i]);
    vector<comp> eval_Pe = FFT(Pe, inversa);
    vector<comp> eval_Po = FFT(Po, inversa);
    vector<comp> eval(n);
    double angulo = 2*PI / n * (inversa? -1 : 1);
    comp w(1), w_n(cos(angulo), sin(angulo));
    for(int i = 0; i < n / 2; ++i){
        eval[i] = eval_Pe[i] + w * eval_Po[i];
        eval[i+n/2] = eval_Pe[i] - w*eval_Po[i];
        if(inversa){eval[i]/=2; eval[i+n/2]/=2;}
        w *= w_n;
    } return eval;
}
```

Multiplicar polinomios. Complejidad: Tiempo $O(n \log n)$ - Memoria extra $O(n \log n)$, donde n es el grado máximo del polinomio A o B.

```cpp
vi multiplicar(vi A, vi B){
    vector<comp> cA(all(A)), cB(all(B));
    int n = 1;
    while(n < sz(A) + sz(B)) n *= 2;
    cA.resize(n);
    cB.resize(n);
    vector<comp> val_A = FFT(cA, false);
    vector<comp> val_B = FFT(cB, false);
    for(int i=0; i<n; ++i) val_A[i] *= val_B[i];
    val_A = FFT(val_A, true);
    vi res(n);
    for(int i = 0; i < n; ++i)
        res[i] = round(val_A[i].real());
    int carry = 0;
    for(int i = 0; i < n; i++){ res[i] += carry;
        carry = res[i] / 10; res[i] %= 10;
    } return res;
}
```

MÉTODOS NUMÉRICOS

▸ Inversa de Matriz con Gauus-Jordan

Complejidad: Tiempo $O\left(n^3\right)$ - Memoria $O\left(n^2\right)$

```cpp
#define eps 0.0001
vector<vector<double>> gauss(vector<vector<double>> m){
    int tam = sz(m);
    vector<vector<double>> id(tam, vector<double>(tam));
    for(int i=0; i<tam; i++) id[i][i] = 1;
    for(int i=0; i<tam; i++){
        // buscar un no 0 para poner en (i,i)
        if(abs(m[i][i]) < eps){
            int j=i+1
            while(abs(m[j][i]) > eps && j<tam){
                swap(m[j], m[i]);
                swap(id[j], id[i]); j++;
            }
            if(j == tam){/*...*/}//no invertible
        }
        /// ponner un uno en (i,i) /// modificar la fila i
        double tmp = m[i][i];
        for(int k=0; k<tam; k++)
            id[i][k] /= tmp, m[i][k] /= tmp;
        /// poner columna i en todo 0, excepto casilla (i,i)
        for(int j=0; j<tam; j++){
            if(j==i) continue;
            //a la fila j le restamos la fila i
            tmp = m[j][i];
            for(int k=0; k<tam; k++){
                m[j][k] = m[j][k]-tmp*m[i][k];
                id[j][k] = id[j][k]-tmp*id[i][k];
            }
        }
    } return id;
}
vector<double> matporvec(vector<vector<double>> &m,
    vector<double> &b){
    vector<double> ans(sz(b));
    for(int i=0; i<sz(b); i++)
    for(int k=0; k<sz(b); k++)
        ans[i] += m[i][k]*b[k];
    return ans;
}
```

▸ Sistemas de ecuaciones lineales

Dado un sistema de n ecuaciones con m incógnitas. Devuelve el numero de soluciones del sistema 0, 1 o ∞ y si existe al menos una solución la guarda en el vector `ans`. Complejidad: Tiempo $O((n+m)nm)$ - Memoria $O(nm)$.

```cpp
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a big number
int gauss(vector<vector<double>> a, vector<double> &ans){
    int n = (int) sz(a);
    int m = (int) sz(a[0]) - 1;
    vi where (m, -1);
    for(int col=0,row=0; col<m && row<n; ++col){
        int sel = row;
        for (int i=row; i<n; ++i)
        if(abs(a[i][col])>abs(a[sel][col]))sel=i;
        if(abs(a[sel][col]) < EPS) continue;
        for(int i=col; i<=m; ++i)
            swap(a[sel][i], a[row][i]);
        where[col] = row;
        for(int i=0; i<n; ++i)
        if(i != row){
            double c = a[i][col] / a[row][col];
            for(int j=col; j<=m; ++j)
                a[i][j] -= a[row][j] * c;
        } ++row;
    } ans.assign (m, 0);
    for(int i=0; i<m; ++i) if(where[i] != -1)
        ans[i] = a[where[i]][m] / a[where[i]][i];
    for(int i=0; i<n; ++i){
        double sum = 0;
        for(int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if(abs(sum - a[i][m]) > EPS) return 0;
    }
    for(int i=0; i<m; ++i) if(where[i] == -1)
        return INF;
    return 1;
}
```

▸ Sistemas de ecuaciones modulo 2

Dado un sistema de n ecuaciones con m incógnitas modulo 2. Devuelve el numero de soluciones del sistema 0, 1 o ∞ y si existe al menos una solución la guarda en el vector `ans`. Complejidad: Tiempo $O((n+m)nm)$ Memoria $O(nm)$.

```cpp
int gauss (vector<bitset<MAXN>> A, int n, int m, bitset<MAXN> &ans){
    // where[i] guarda el indice de la ecaucion donde se puede despejar variable i, -1 si es independiente
    vi where(m, -1);
    //  Matriz "diagonal"
    for(int col=0,row=0; col<m && row<n; ++col){
        // busca pivote
        for(int i = row; i<n; ++i) if(A[i][col]){
            swap (A[i], A[row]);
            break;
        } if(!A[row][col]) continue;
        where[col] = row;
        // poner 0 todos lados
        for(int i=0; i<n; ++i)
        if(i != row && A[i][col]) A[i] ^= A[row];
        ++row;
    }
    // sacar respuesta // variables independientes = 1, t
    for(int i = 0; i < m; ++i)
        ans[i] = where[i] == -1 ? 1 : ((A[where[i]].count() ^ 1) & 1);
    // si hay alguna variable libre
    for(int i = 0; i < m; ++i) if(where[i] == -1)
        return INF;
    return 1;
}
```

SPARSE TABLE

Complejidad: Tiempo de precalculo $O(n \log n)$ - Tiempo en responder $O(\log(r-l+1))$ - Tiempo en responder para operaciones idempotentes $O(1)$ - Memoria extra $O(n \log n)$. `LOGN` es $\lceil \log_2(\texttt{MAXN}) \rceil$. Indexado en 0.

```
struct sparse_table{
    int n, NEUTRO;
    vvi ST;
    vi lg2;
    int f(int a, int b){return a + b;}
    sparse_table(int _n, int data[]){
        n = _n;
        NEUTRO = 0;
        lg2.resize(n + 1);
        lg2[1] = 0;
        for(int i=2; i<=n; ++i)lg2[i]=lg2[i/2]+1;
        ST.resize(lg2[n] + 1, vi(n + 1, NEUTRO));
        for(int i=0; i<n; ++i)ST[0][i] = data[i];
        for(int k = 1; k <= lg2[n]; ++k){
            int fin = (1 << k) - 1;
            for(int i = 0; i + fin < n; ++i)
            ST[k][i]=f(ST[k-1][i],
                       ST[k-1][i+(1<<(k-1))]);
        }
    }
    int query(int l, int r){
        if(l > r) return NEUTRO;
        int ans = NEUTRO;
        for(int k = lg2[n]; 0 <= k; --k){
            if( r - l + 1 < (1 << k) ) continue;
            ans = f(ans, ST[k][l]);
            l += 1 << k;
        }
        return ans;
    }
    int queryIdem(int l, int r){
        if(l > r) return NEUTRO;
        int lg = lg2[r - l + 1];
        return f(ST[lg][l], ST[lg][r-(1<<lg)+1]);
    }
};
```

FENWICK TREE

T: $O(\log n)$ | **M:** $O(n)$ | *Indexado en 1*

BIT.cpp

```
#define lsb(S) ((S) & (-S))
struct fenwick_tree{
    int n;
    vi BIT;
    fenwick_tree(int _n): n(_n){BIT.resize(n + 1);}
    void add(int pos, int x){
        while(pos <= n){
            BIT[pos] += x;
            pos += lsb(pos);
        }
    }
    int sum(int pos){
        int res = 0;
        while(pos){
            res += BIT[pos];
            pos -= lsb(pos);
        } return res;
    }
};
```

SEGMENT TREE

```
struct node{
    int val, lazy;
    node():val(0), lazy(0){}
    node(int x, int lz = 0):val(x), lazy(lz){}
    const node operator+(const node &b)const{
        return node(val + b.val);
    }
}
```

▶ Actualizaciones puntuales

Complejidad: Tiempo de precalculo $O(n)$ - Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$. Indexado en 1.

```
struct segment_tree{
    struct node{...};
    vector<node> nodes;
    segment_tree(int n, int data[]){
        nodes.resize(4 * n + 1);
        build(1, n, data);
    }
    void build(int left, int right, int data[], int pos = 1){
        if(left == right){
            nodes[pos] = node(data[left]);
            return;
        }
        int mid = (left + right) / 2;
        build(left, mid, data, pos * 2);
        build(mid + 1, right, data, pos * 2 + 1);
        nodes[pos] = nodes[pos*2]+nodes[pos*2+1];
    }
    void update(int x, int idx, int left, int right, int pos =
    ↪  1){
        if(idx < left || right < idx) return;
        if(left == right){
            nodes[pos].val += x;
            return;
        }
        int mid = (left + right) / 2;
        update(x, idx, left, mid, pos * 2);
        update(x, idx, mid+1, right, pos*2+1);
        nodes[pos] = nodes[pos*2]+nodes[pos*2+1];
    }
    node query(int l, int r, int left, int right, int pos = 1){
        if(r < left || right < l) return node();
        if(l <= left && right <= r)
            return nodes[pos];
        int mid = (left + right) / 2;
        return query(l, r, left, mid, pos*2) + query(l, r,
        ↪  mid+1, right, pos*2+1);
    }
};
```

▶ Actualizaciones sobre rangos

Complejidad: Tiempo de precalculo $O(n)$ - Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$.

```
struct segment_tree{
    struct node{...};
    vector<node> nodes;
    segment_tree(int n, int data[]){...}
    void build(...){...}
    void combine_lz(int lz, int pos){nodes[pos].lazy += lz;}
    void apply_lz(int pos, int tam){
        nodes[pos].val += nodes[pos].lazy * tam;
        nodes[pos].lazy = 0;
    }
    void push_lz(int pos, int left, int right){
        int len = abs(right - left + 1);
        if(1 < len){
            combine_lz(nodes[pos].lazy, pos*2);
            combine_lz(nodes[pos].lazy, pos*2+1);
        } apply_lz(pos, len);
    }
    void update(int x, int l, int r, int left, int right, int
    ↪  pos = 1){
        push_lz(pos, left, right);
        if(r < left || right < l) return;
        if(l <= left && right <= r){
            combine_lazy(x, pos);
            push_lazy(pos, left, right);
            return;
        }
        int mid = (left + right) / 2;
        update(x, l, r, left, mid, pos * 2);
        update(x, l, r, mid+1, right, pos*2+1);
        nodes[pos] = nodes[pos*2]+nodes[pos*2+1];
    }
    node query(int l, int r, int left, int right, int pos = 1){
        push_lz(pos, left, right);
        ...
    }
};
```

▶ Sparse segment tree

```
struct node{
    ll lazy, maxi, sum;
    node *left = nullptr, *right = nullptr;
    node(): lazy(0), maxi(0), sum(0){}
    void extend(){
        if(left) return;
        left = new node;
        right = new node;
    }
    void combine_lz(ll lz){
        lazy += lz;
    }
    void apply_lz(ll len){
        sum += len * lazy; lazy = 0;}
    void push_lz(ll L, ll R){
        int len = R - L + 1;
        ll mid = L + (R - L) / 2;
        if(len){
            extend();
            left->combine_lz(lazy);
            right->combine_lz(lazy);
        }
        apply_lz(len);
    }
    void update(ll x, ll l, ll r, ll L, ll R){
        push_lz(L, R);
        if(r < L || R < l) return;
        if(l <= L && R <= r){
            combine_lz(x);
            push_lz(L, R);
            return;
        }
        ll mid = L + (R - L) / 2;
        extend();
        left->update(x, l, r, L, mid);
        right->update(x, l, r, mid + 1, R);
        sum = left->sum + right->sum;
    }
    ll query(ll l, ll r, ll L, ll R){
        push_lz(L, R);
        if(r < L || R < l) return 0;
        if(l <= L && R <= r) return sum;
        extend();
        ll mid = L + (R - L) / 2;
        return left->query(l, r, L, mid) + right->query(l, r,
        ↪  mid + 1, R);
    }
};
```

SQRT DECOMPOSITION

▶ Algoritmo de MO

Complejidad: Tiempo en responder $O((n+q)\sqrt{n}F + q \log q)$, donde $O(F)$ es la complejidad de `add()` y `remove()`.

```
const int block_size = 300;
struct query {
    int l, r;
    int block, i;
    bool operator<(const query &b) const {
        if(block == b.block) return r < b.r;
        return block < b.block;
    }
```

```
};
void add(int idx){/// TO-DO}
void remove(int idx){/// TO-DO}
int get_answer(){return 0; /// TO-DO}
vi solve(vector<query> &queries) {
    vi answers(sz(queries));
    sort(queries.begin(), queries.end());
    int cur_l = 0, cur_r = -1;
    for(query q : queries){
        while(cur_l > q.l) add(--cur_l);
        while(cur_r < q.r) add(++cur_r);
        while(cur_l < q.l) remove(cur_l++);
        while(cur_r > q.r) remove(cur_r--);
        answers[q.i] = get_answer();
    } return answers;
}
```

DSU

Complejidad: Tiempo $O(\log(n))$ - Memoria $O(n)$, donde n es la cantidad total de elementos. La complejidad temporal es por cada función.

`P[MAXN]`: guarda el representante para cada nodo.

`RA[MAXN]`: guarda el rango (peso) del conjunto de cada representante para el *small to large*.

```
struct dsu{
    struct action{
        int x_p, y_p, rank_y;
    };
    vi RA, P;
    vector<action> actions;
    dsu(int n){
        RA.resize(n, 1);
        P.resize(n);
        iota(P.begin(), P.end(), 0);
    }
    int root(int x){
        return x == P[x] ? x : P[x] = root(P[x]);
    }
    bool join(int x, int y, bool recording){
        x = root(x);
        y = root(y);
        if(x == y) return false;
        if(RA[x] >= RA[y]) swap(x, y);
        if(recording) actions.pb({x, y, RA[y]});
        RA[y] += RA[x];
        P[x] = y;
        return true;
    }
    void rollback(int cnt){
        while(cnt-- > 0 && actions.size()){
            action act = actions.back();
            actions.pop_back();
            RA[act.y_p] = act.rank_y;
            P[act.x_p] = act.x_p;
        }
    }
};
```

GRAFOS

```
struct edge{
    int from, to;
    ll w;
    const bool operator<(const edge &b)const{
        return w > b.w;
    }
};
struct pos{
    int from;
    ll c;
    const bool operator<(const pos &b)const{
        return c > b.c;
    }
};
```

▶ Caminos mínimos

Dijkstra. Complejidad: Tiempo $O(|E|\log|V|)$ - Memoria extra $O(|E|)$.

```
ll dijkstra(int a, int b, vector<edge> graph[]){
    ll dist[MAXN];
    bool vis[MAXN] = {};
    fill(dist, dist + MAXN, LLONG_MAX);
    priority_queue<pos> q;
    q.push(pos{a, 0}); dist[a] = 0;
    while(sz(q)){
        pos cur = q.top(); q.pop();
        if(vis[cur.from]) continue;
        vis[cur.from] = true;
        for(edge &e : graph[cur.from]){
            if(dist[e.to] <= dist[e.from] + e.w) continue;
            dist[e.to] = dist[e.from] + e.w;
            q.push(pos{e.to, dist[e.to]});
        }
    } return dist[b];
}
```

Bellman-Ford. Complejidad: $O(|V||E|)$.

```
vi bellman_ford(int s, int n, vector<edge> &edges, bool cycles
  ↪  = false){
    vi d(n, (cycles ? 0 : INT_MAX));
    d[s] = 0;
    vi P(n, -1); /// Predecesor
    for(int i = 0; i < n - 1; ++i)
    for(edge &e : edges){
        if(d[e.from] == INT_MAX) continue;
        if(d[e.to] > d[e.from] + e.w){
            d[e.to] = d[e.from] + e.w;
            P[e.to] = e.from;
        }
    }
    int last_relax = -1;
    for(edge &e : edges){
        if(d[e.from] == INT_MAX) continue;
        if(d[e.to] > d[e.from] + e.w){
            d[e.to] = d[e.from] + e.w;
            P[e.to] = e.from;
            last_relax = e.to;
        }
    }
    if(last_relax == -1) return d;
    return {}; /// VACIO
}
```

Floyd-Warshall. Complejidad: $O\left(|V|^3\right)$.

```
vvi floyd_warshall(int n){
    const int INF = INT_MAX; // para evitar overflow después
    vvi d(n, vi(n, INF));
    /// aqui inicializa con la lista/matriz de adyacencia
    /// luego calcula la dp
    for(int k = 0; k < n; ++k){
        for(int i = 0; i < n; ++i){
            for(int j = 0; j < n; ++j){
                if(d[i][k]==INF || d[k][j]==INF)
                    continue;
                if(d[i][j] > d[i][k] + d[k][j]) d[i][j] =
                ↪  d[i][k] + d[k][j];
            }
        }
    } return d;
}
```

Johnson's algorithm. Complejidad: $O(|V||E|\log|V|)$. Sea $p : V \to \mathbb{R}$ una función potencial del grafo. El algoritmo es como sigue:

1. Hacemos una transformación en el grafo cambiando los pesos w a $w'(u,v) = w(u,v) + p(u) - p(v)$.
2. Calculamos la distancia mínima $d' : V \times V \to \mathbb{R}$ desde cada nodo a todos los demás con Dijkstra.
3. Finalmente, la distancia mínima de u a v en el grafo original es $d(u,v) = d'(u,v) - p(u) + p(v)$.

La función potencial p puede ser cualquiera. Usando Bellman-Ford se puede calcular el potencial $p(u)$ como el camino más corto que termina (o empieza) en u.

▶ Árboles

Prim. Complejidad: Tiempo $O(|E|\log|V|)$. `eCost[MAXN]` es el arreglo de costos mínimos de cada nodo para incluirlo en el MST.

```
ll prim(vector<edge> graph[]){
    ll e_cost[MAXN], ans = 0;
    bool vis[MAXN] = {};
    fill(e_cost, e_cost + MAXN, LLONG_MAX);
    priority_queue<edge> q;
    q.push(edge{1, 1, 0});
    while(sz(q)){
        int node = q.top().to;
        ll w = q.top().w; q.pop();
        if(vis[node]) continue; vis[node] = true;
        for(edge &e : graph[node]){
            if(vis[e.to] || e_cost[e.to] <= e.w) continue;
            e_cost[e.to] = e.w;
            q.push(e);
        } ans += w;
    } return ans;
}
```

Kruskal. Complejidad: Tiempo $O\left(|E|\log|E|\right)$.

```
ll kruskal(vector<edge> &edges, int n){
    sort(all(edges));
    dsu mset(n);
    ll res = 0;
    for(edge &e : edges){
        if(mset.root(e.from) == mset.root(e.to)) continue;
        mset.join(e.from, e.to);
        res += e.w;
    }
    return res;
}
```

Boruvka. Complejidad: Tiempo $O\left(|E|\log|V|\right)$. $|V| = n$. `dsu.join()` devuelve `true` si la unión se llevó a cabo o `false` en otro caso.

```
ll boruvka(vector<edge> &edges, int n){
    dsu mset(n);
    int min_edge[n];
    ll res = 0;
    while(mset.cnt_comp > 1){
        fill(min_edge, min_edge + n, -1);
        for(int i = 0; i < sz(edges); ++i){
            int u = mset.root(edges[i].from);
            int v = mset.root(edges[i].to);
            if(u == v) continue;
            if(min_edge[u] == -1 || edges[i].w <
            ↪  edges[min_edge[u]].w) min_edge[u] = i;
            if(min_edge[v] == -1 || edges[i].w <
            ↪  edges[min_edge[v]].w) min_edge[v] = i;
        }
        for(int i = 0; i < n; ++i){
            int idx_e = min_edge[i];
            if(idx_e == -1) continue;
            res += mset.join(edges[idx_e].from,
            ↪  edges[idx_e].to) * edges[idx_e].w;
        }
    } return res;
}
```

MST dirigido. Complejidad: $O(|E|\log|V|)$. AGREGAR PEQUEÑA DESCRIPCIÓN.

```cpp
/// MEJORAR ESTA COSA, SOLO LO COPIE Y PEGUÉ porcuestionesdetiempo
const int N = 3e5 + 9;
const ll inf = 1e18;
template<typename T> struct PQ {
    ll sum = 0;
    priority_queue<T, vector<T>, greater<T>> Q;
    void push(T x) { x.w -= sum; Q.push(x); }
    T pop() { auto ans = Q.top(); Q.pop(); ans.w += sum; return ans; }
    int size() { return sz(Q); }
    void add(ll x) { sum += x; }
    void merge(PQ &x){
        if (size() < sz(x)){
            swap(sum, x.sum);
            Q.swap(x.Q);
        }
        while(sz(x)){
            auto tmp = x.pop();
            tmp.w -= sum;
            Q.push(tmp);
        }
    }
};
struct edge {
    int u, v; ll w;
    bool operator > (const edge &rhs) const { return w > rhs.w; }
};
struct DSU {
    vi par;
    DSU (int n) : par(n, -1) {}
    int root(int i) { return par[i] < 0 ? i : par[i] = root(par[i]); }
    void set_par(int c, int p) { par[c] = p; }
};
// returns parents of each vertex
// each edge should be distinct
// it assumes that a solution exists (all vertices are reachable from root)
// 0 indexed
// Takes ~300ms for n = 2e5
vi DMST(int n, int root, const vector<edge> &edges) {
    vi u(2 * n - 1, -1), par(2 * n - 1, -1);
    edge par_edge[2 * n - 1];
    vi child[2 * n - 1];
    PQ<edge> Q[2 * n - 1];
    for(auto e : edges) Q[e.v].push(e);
    for(int i = 0; i < n; i++)
        Q[(i+1) % n].push({i, (i+1) % n, inf});
    int super = n;
    DSU dsu(2 * n - 1);
    int head = 0;
    while(sz(Q[head])) {
        auto x = Q[head].pop();
        int nxt_root = dsu.root(x.u);
        if (nxt_root == head) continue;
        u[head] = nxt_root;
        par_edge[head] = x;
        if (u[nxt_root] == -1) head = nxt_root;
        else {
            int j = nxt_root;
            do {
                Q[j].add(-par_edge[j].w);
                //Q[super].merge(move(Q[j]));
                Q[super].merge(Q[j]);
                assert(u[j] != -1);
                child[super].pb(j);
                j = dsu.root(u[j]);
            } while (j != nxt_root);
            for(auto u : child[super]) par[u] = super, dsu.set_par(u, super);
            head = super++;
        }
    }
    vi res(2 * n - 1, -1);
    queue<int> q; q.push(root);
    while (sz(q)) {
        int u = q.front();
        q.pop();
        while (par[u] != -1) {
            for (auto v : child[par[u]]) {
                if (v != u) {
                    res[par_edge[v].v] = par_edge[v].u;
                    q.push(par_edge[v].v);
                    par[v] = -1;
                }
            }
            u = par[u];
        }
    }
    res[root] = root; res.resize(n);
    return res;
}
int main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    int n, m, root; cin >> n >> m >> root;
    vector<edge> edges(m);
    for(auto &e : edges) cin >> e.u >> e.v >> e.w;
    auto res = DMST(n, root, edges);
    unordered_map<int, int> W[n];
    for (auto u : edges) W[u.v][u.u] = u.w;
    ll ans = 0;
    for(int i = 0; i < n; i++) if(i != root)
        ans += W[i][res[i]];
    cout << ans << '\n';
    for(auto x : res) cout << x << ' ';
    cout << '\n';
}
```

Block-Cut Tree Complejidad: $O(n)$?

```cpp
/// MEJORAR ESTA COSA, SOLO LO COPIE Y PEGUÉ porcuestionesdetiempo
vvi comps;//nodes in each component
vvi bcc;//nodes to components that it belong
vi st;//internal stack
vi low, num;
int id;
int sz;
void dfs_biconnected(vvi &g, int u, int pre){
  low[u] = num[u] = ++id;
  st.pb(u);
  for(auto v: g[u]) {
    if(!num[v]) {
      dfs_biconnected(g, v, u);
      low[u] = min(low[u], low[v]);
      if(low[v] >= num[u]) {
        int x;
        comps.pb(vi());
        do {
          x = st.back();
          st.pop_back();
          bcc[x].pb(sz);
          comps.back().pb(x);
        } while(x ^ v);
        bcc[u].pb(sz);
        comps.back().pb(u);
        sz++;
      }
    } else if(v != pre) low[u] = min(low[u], num[v]);
  }
}
vi utoubt;//its component or its AP index
vb uisart;//u is AP?
vvi bt;//block cut tree
void generateBlockCutTree(vvi &g){
    int n = sz(g);
    sz = id = 0;
    bcc.resize(0); low.resize(0); num.resize(0);
    bcc.resize(n); low.resize(n); num.resize(n);
    dfs_biconnected(g, 0, 0);
    bt.resize(sz);
    utoubt.resize(0); utoubt.resize(n);
    uisart.resize(0); uisart.resize(n);
    for(int u = 0; u < n; u++){
        if(sz(bcc[u]) > 1) { //Articulation
            utoubt[u] = sz++;
            uisart[u] = true;
            bt.pb(vi());
            for(auto v: bcc[u]){
                bt[utoubt[u]].pb(v);
                bt[v].pb(utoubt[u]);
            }
        } else //Not articulation point
            utoubt[u] = bcc[u][0];
    }
}
```

T: $O(n \log n)$ | **M:** $O(n \log n)$ | *LCA y n-ésimo ancestro. Precalcula 2^k-ancestros y profundidades. 1-index.*

lca.cpp

```cpp
void precalc(int u, int p = 0){
    depth[u] = depth[p] + 1; P[0][u] = p;
    rep(k, 1, LOGN + 1) P[k][u] = P[k - 1][P[k - 1][u]];
    for(int v : T[u]) if(v != p) precalc(v, u);
}
int LCA(int a, int b){
    if(depth[b] < depth[a]) swap(a, b);
    int diff = depth[b] - depth[a];
    repr(k, 0, LOGN + 1) if(is_on(diff, k)) b = P[k][b];
    if(a == b) return a;
    repr(k, 0, LOGN + 1) if(P[k][a] != P[k][b]){
        a = P[k][a];
        b = P[k][b];
    } return P[0][a];
}
int nth_ancestor(int u, int n){
    repr(k, 0, LOGN + 1) if(is_on(n, k)) u = P[k][u];
    return u;
}
```

T: $O(n \log n)$ | **M:** $O(n)$ | *Sack. Consultas en todos los subárboles. 1-index. add agrega/elimina un subárbol. Responder después de add.*

sack.cpp

```cpp
int precalc(int u, int p = 0){
    depth[u] = depth[p] + 1;
    for(int v : T[u]) if(v != p) sub[u] += precalc(v, u);
    return ++sub[u];
}
void add(int u, int x, int p = 0){
    for(int v: T[u]) if(v != p && !big[v]) add(v, x, u);
}
void dfs(int u, bool keep, int p = 0){
    int maxi = -1, big_child = -1;
    for(int v : T[u]) if(v != p && sub[v] > maxi)
        maxi = sub[v], big_child = v;
    for(int v : T[u]) if(v!=p && v!=big_child) dfs(v, 0, u);
    if(big_child!=-1) dfs(big_child,1,u), big[big_child] = 1;
    add(u, 1, p);
    if(big_child != -1) big[big_child] = 0;
    if(!keep) add(u, -1, p);
}
```

T: $O(n \log n)$ | **M:** $O(n)$ | *Centroid Decomposition. Para constuir el árbol de centroides usar la arista `(p, centroid)` en build.*

centroid.cpp

```cpp
struct CT{
    vvi T;
    vi sub;
    vector<bool> removed;
    CT(vvi &_T) : T(_T){
        sub.resize(sz(T)); removed.resize(sz(T)); build(0);
    }
    int find_sub(int u, int p = -1){
        for(int v : T[u]) if(v != p && !removed[v])
            sub[u] += find_sub(v, u);
        return ++sub[u];
    }
    int find_centroid(int u, int n, int p = -1){
        for(int v:T[u])if(v!=p && sub[v]>n/2 && !removed[v])
            return find_centroid(v, n, u);
```

```
16          return u;
17      }
18      void build(int u, int p = -1){
19          int n = find_sub(u, p);
20          int centroid = find_centroid(u, n, p);
21          if(p == -1) p = centroid;
22          removed[centroid] = 1;
23          vi adj(all(T[centroid]));
24          for(int v : adj) if(!removed[v]) build(v, centroid);
25      }
26  };
```

► Matching

Algunos problemas de flujos

- *Maximum Weight Closure.* Sea N_1 una clausura de G y $N_2 = V \setminus N_1$, tenemos que $w(N_1) = \sum_{i\in N_1^+} w_i - \sum_{i\in N_1^-} |w_i|$ y $Cap.Corte = \sum_{i\in N_2^+} w_i + \sum_{i\in N_1^-} |w_i|$. Entonces
$$Cap.Corte + w(N_1) = \sum_{i\in N_1^+} w_i + \sum_{i\in N_2^+} w_i.$$
- *Mínima cobertura de vértices.* En grafos generales es NP-Completo. En grafos bipartitos el máximo emparejamiento es igual al numero de vertices en la mínima cobertura. Para el problema con pesos en los nodos, unimos s a todos los nodos en L con capacidad igual al peso de cada nodo, unimos los nodos de R a t de la misma manera y unimos los nodos de L a R con capacidad infinita. El máximo flujo es el peso mínimo de la mínima cobertura.
- *Máximo conjunto independiente.* Cualquier conjunto independiente es el complemento de alguna cobertura de vértices.
- *Mínimo cubrimiento de caminos independientes.* En grafos generales es NP-hard. En DAG's duplicamos los nodos en un lado IN y un lado OUT. Conectamos s al lado OUT y el lado IN a t. Las aristas del DAG las agregamos del lado OUT al lado IN. Sea M el máximo emparejamiento de la red anterior, entonces el mínimo cubrimiento es $|V| - M$.
- *Mínimo cubrimiento de caminos NO necesariamente independientes.* En grafos generales es NP-hard. En DAG's transformamos el DAG a su clausura transitiva y aplicamos el problema anterior.
- *Teorema de Menger.* Sea G un grafo finito (sin pesos) con vértices s, t distintos. El corte mínimo (por aristas) entre s y t es igual a la cantidad máxima de caminos independientes por aristas de s a t. Si s y t no son adyacentes, el corte mínimo por vértices entre s y t es igual a la cantidad máxima de caminos independientes por vértices de s a t.
- *Teorema de Mirsky.* En todo POSET, el tamaño de la cadena de mayor tamaño es igual al número de anticadenas necesarias para cubrir todos los elementos del conjunto.
- *Teorema de Dilworth.* En todo POSET, el tamaño de la anticadena de mayor tamaño es igual al número de cadenas necesarias para cubrir todos los elementos del conjunto.

Teoremas de Matching

Notacion: Sea $G = (L \cup R, E)$ bipartito. Para $S \subseteq L$ $N(S) = \{\, r \in R : \exists\, s \in S,\ \{s, r\} \in E \,\}$. $\nu(G)$ = tamaño de un matching máximo.

Teorema de Hall. Existe un matching que satura a L Si y Solo Si $|N(S)| \geq |S|$ para todo $S \subseteq L$.

Deficiencia (König–Ore). Se $\delta(G) = \max_{S\subseteq L}\big(|S| - |N(S)|\big) \geq 0$. Importante notar que $S = \varnothing$ da 0, así que $\delta \geq 0$. El teorema dice
$$\nu(G) \;=\; |L| - \delta(G),$$
Asi $\delta(G)$ es el número de vértices de L que quedan sin cubrir en un matching máximo.

Hall caso $\delta = 0$. Matching perfecto $\iff |L| = |R|$ y $\delta(G) = 0$.

Notación para multiplicidades. Sea $G = (L \cup R, E)$ bipartito, con $a : L \to \mathbb{Z}_{\geq 0}$ (oferta), $b : R \to \mathbb{Z}_{\geq 0}$ (demanda) y $u : L \times R \to \mathbb{Z}_{\geq 0} \cup \{\infty\}$ (capacidad por arista), con $u_{ij} = 0$ si $\{i, j\} \notin E$. Para $A \subseteq L$ se define $a(A) = \sum_{i\in A} a_i$, y análogamente $b(B)$ para $B \subseteq R$. Para $X \subseteq L$, $Y \subseteq R$ se define $u(X, Y) = \sum_{i\in X,\, j\in Y} u_{ij}$. $N(S) \subseteq R$ es la vecindad de $S \subseteq L$ en G. Cuando no hace falta distinguir lados, se escribe $c : L \cup R \to \mathbb{Z}_{\geq 0}$ y $c(A) = \sum_{v\in A} c_v$.

Hall con multiplicidades. Sea G_c el grafo *expandido*: cada v se reemplaza por c_v copias, y dos copias son adyacentes $\iff$ sus originales lo son (equivale a $u \equiv \infty$ sobre E). Sea L_c el conjunto de copias de L. Entonces

G_c tiene un matching que satura $L_c \iff c\big(N(S)\big) \geq c(S)\, \forall\, S \subseteq L$.

Deficiencia con multiplicidades. Se cumple
$$\nu(G_c) = c(L) - \max_{S\subseteq L}\big(c(S) - c(N(S))\big).$$
G_c tiene matching perfecto $\iff c(L) = c(R)$ y se cumple la condición de Hall anterior.

Transporte con capacidades. Existe $x : E \to \mathbb{Z}_{\geq 0}$ con $0 \leq x_{ij} \leq u_{ij}$, $\sum_j x_{ij} = a_i$ para todo $i \in L$ y $\sum_i x_{ij} \leq b_j$ para todo $j \in R$ (es decir, se *satura* L) si y solo si $a(S) \leq b(T) + u(S, R \setminus T)$ Si y Solo Si $S \subseteq L$, $T \subseteq R$. Si además $a(L) = b(R)$, también se satura R (igualdad en todas las columnas).

Red y cortes. Aristas $s \to i$ con capacidad a_i, $i \to j$ con capacidad u_{ij}, y $j \to t$ con capacidad b_j. Un corte s–t se describe por $S \subseteq L$, $T \subseteq R$: el lado de s es $\{s\} \cup S \cup T$ y el lado de t es $\{t\} \cup (L \setminus S) \cup (R \setminus T)$. Su capacidad es $a(L \setminus S) + u(S, R \setminus T) + b(T)$. Por max-flow min-cut, el flujo máximo vale $a(L)$ (se satura L) $\iff$ todo corte tiene capacidad $\geq a(L)$.

Matching con multiplicidades y restricciones. El valor del máximo emparejamiento (con cada $i \in L$ usado hasta a_i veces y cada $j \in R$ hasta b_j veces, con restriccione u_{ij}) es $a(L)$, es decir, se *satura* L, si y solo si todo corte s–t tiene capacidad al menos $a(L)$. Esto es
$$a(S) \;\leq\; b(T) \;+\; u(S, R \setminus T) \qquad \forall\, S \subseteq L,\ T \subseteq R.$$
Si además $a(L) = b(R)$, también se satura R, y el emparejamiento es perfecto.

Caso $u \equiv \infty$ en E (Hall con pesos). El término $u(S, R \setminus T)$ vale ∞ salvo que $T \supseteq N(S)$, y entre esos T el que minimiza $b(T)$ es $T = N(S)$. La condición queda $a(S) \leq b\big(N(S)\big)\, \forall\, S \subseteq L$.

Teorema de Gale–Ryser. Sean $a_1 \geq a_2 \geq \cdots \geq a_m \geq 0$ y $b_1, \ldots, b_n \geq 0$ enteros. Existe una matriz $A \in \{0, 1\}^{m\times n}$ con suma a_i en la fila i y suma b_j en la columna j
$$\iff \quad \sum_{i=1}^{m} a_i = \sum_{j=1}^{n} b_j \quad \text{y} \quad \sum_{i=1}^{k} a_i \;\leq\; \sum_{j=1}^{n} \min(b_j, k) \qquad \forall\, k \in [m].$$
Forma de dominancia: con $b_k^* = \#\{j : b_j \geq k\}$ (partición conjugada), $\sum_j \min(b_j, k) = \sum_{i\leq k} b_i^*$, así que la condición es $a \preceq b^*$.

Prueba (desde el corte). Es el caso $G = K_{m,n}$, $u \equiv 1$. La capacidad del corte es $a(L \setminus S) + |S|\,|R \setminus T| + b(T)$, luego la condición es $a(S) \leq b(T) + |S|\,|R \setminus T|$. Con $|S| = k$ fijo, el lado derecho se minimiza columna a columna (b_j si $j \in T$, k si no), dando $\sum_j \min(b_j, k)$; el lado izquierdo se maximiza tomando los k mayores a_i. ■

Contraejemplos (límites). $a = (2, 2)$, $b = (4, 0)$: las sumas coinciden pero para $k = 1$ se tiene $2 \leq \min(4, 1) + \min(0, 1) = 1$, falso. $a = (1)$, $b = (2)$ cumple las desigualdades con $k = 1$ pero no la igualdad de sumas.

Aclaración de alcance. Gale–Ryser es para *dos* conjuntos (filas, columnas): las entradas (i, j) y (j, i) son independientes. Si el mismo conjunto de elementos aparece en ambos lados y cada par $\{i, j\}$ se usa a lo sumo una vez, el objeto cambia: un grafo simple con secuencia de grados dada (matriz simétrica con diagonal cero; Erdős–Gallai), o un torneo (matriz con $x_{ij} + x_{ji} = 1$ para $i \neq j$; Landau). Las condiciones de existencia son otras.

Teorema de Landau Sea $S = (b_1, b_2, \ldots, b_n)$ con $b_1 \leq b_2 \leq \cdots \leq b_n$. La secuencia S es la secuencia de salidas de un torneo simple $T(V, E)$ con $|V| = n$ Si y Solo Si
$$\sum_{i=1}^{n} b_i = \binom{n}{2} \quad \text{y} \quad \sum_{i=1}^{k} b_i \geq \binom{k}{2}\ \forall k \in [n-1]$$

► Máximo flujo

Edmonds-Karp Complejidad: Ford-Fulkerson $O(|E|\cdot F)$, Edmonds-Karp $O(|V||E|^2)$.

```
struct edge {
    int from, to;
    ll w, c, f;
    // weight, capacity, flow
};
class ford_fulkerson {
public:
    ford_fulkerson (vector<vector<edge>> &graph) :
    ↪  graph(graph){}
    ll get_max_flow(int s, int t){
        init();
        ll f = 0;
        while(find_and_update(s, t, f)){}
        return f;
    }
    vi get_st_cut(const int &s){
        bool vis[sz(graph)] = {};
        vi S;
        queue<int> q;
        q.push(s);
        S.pb(s);
        vis[s] = true;
        while(sz(q)){
            int u = q.front(); q.pop();
            for(int eI : edge_indexes[u]){
                if(edges[eI].c > edges[eI].f &&
                ↪  !vis[edges[eI].to]){
                    q.push(edges[eI].to);
                    vis[edges[eI].to] = true;
                    S.pb(edges[eI].to);
                }
            }
        }
        return S;
    }
    vector<vector<edge>> get_residual_graph(){
        vector<vector<edge>> residual(sz(graph));
        for(int i=0; i<sz(edges); i+=2){
            const edge& e = edges[i];
```

```
            if(e.c > 0){
                residual[e.from].pb({e.from, e.to, e.w, e.c -
                ↪ e.f, e.f});
                residual[e.to].pb({e.to, e.from, -e.w, e.f,
                ↪ -e.f});
            }
        } return residual;
    }
private:
    vector<vector<edge>> graph;
    vector<edge> edges;
    vvi edge_indexes;
    void init(){
      edges.clear();
      edge_indexes.clear();
      edge_indexes.resize(sz(graph));
      for(int u = 0; u < sz(graph); u++){
         for(edge &e : graph[u]){
           edges.pb({u, e.to, e.w, e.c, 0});
           edges.pb({e.to, u, -e.w, 0, 0});
           edge_indexes[u].pb(sz(edges)-2);
           edge_indexes[e.to].pb(sz(edges)-1);
         }
      }
    }
    bool find_and_update(int s, int t, ll &flow){
        queue<int> q;
        // Desde donde llego y con que arista
        vpii from(sz(graph), {-1, -1});
        q.push(s);
        from[s] = {s, -1};
        bool found = 0;
        while(sz(q) && !found){
            int u = q.front(); q.pop();
            for(int eI : edge_indexes[u]){
                if(edges[eI].c > edges[eI].f &&
                  from[edges[eI].to].fi== -1){
                    from[edges[eI].to] = {u, eI};
                    q.push(edges[eI].to);
                    if(edges[eI].to == t)found=1;
                }
            }
        }
        if(!found) return false;
        ll u_flow = LLONG_MAX;
        int cur = t;
        while(cur != s) {
            u_flow = min(u_flow, edges[from[cur].se].c -
            ↪ edges[from[cur].se].f);
            cur = from[cur].fi;
        }
        cur = t;
        while(cur != s){
            edges[from[cur].se].f += u_flow;
            edges[from[cur].se^1].f -= u_flow;
            cur = from[cur].fi;
        }
        flow += u_flow;
        return 1;
    }
};
```

Dinic. Complejidad: $O(|V|^2|E|)$. CAMBIAR A LA VARIANTE CON DFS *Multi-path augmentation*.

```
const int MAXV = 32767; /// 2^15 - 1
template<class T = ll> struct dinic{
    dinic(short V){this->V = V;}
    const static bool SCALING = 1;
    bool sorted = 0;
    short s, t, V;
    int lim = 1; /// Para escalado
    const T INF = numeric_limits<T>::max();
    short level[MAXV]; /// distancia desde s
    short ptr[MAXV]; /// arista que va explorando
    struct edge{
        short to, rev;
        T cap, flow, mcap;
        bool operator<(const edge &b)const{return mcap >
        ↪ b.mcap;}
    };
    vector<edge> adj[MAXV];
    vi adj_cur[MAXV]; /// aristas del grafo de nivel
    void add_edge(short u, short v, T cap, bool is_directed =
    ↪ 1){
        if(u == v) return;
        T add = (is_directed ? 0 : cap);
        adj[u].pb({v, sz(adj[v]), cap, 0 ,cap+add});
        adj[v].pb({u, sz(adj[u])-1, add, 0, cap+add});
    }
    void mysort(){
        if(sorted) return;
        sorted = 1;
        for(int i = 0; i < V; ++i){
            sort(all(adj[i]));
            for(int j=0; j<sz(adj[i]); ++j)
            adj[adj[i][j].to][adj[i][j].rev].rev = j;
        }
    }
    bool bfs(){
        for(int i = 0; i < V; ++i){
            adj_cur[i].clear();
            adj_cur[i].reserve(sz(adj[i]));
        }
        queue<short> q;
        q.push(s);
        fill(level, level + V, -1);
        level[s] = 0;
        while(sz(q)){
            short u = q.front(); q.pop();
            if(u == t) return 1;
            for(int i=0; i < sz(adj[u]); ++i){
                edge &e = adj[u][i];
                if(e.mcap < lim) break;
                if(level[e.to] == -1 && e.cap - e.flow >= lim){
                    level[e.to] = level[u] + 1;
                    adj_cur[u].pb(i);
                    q.push(e.to);
                } else if(level[e.to]==level[u]+1 && e.cap -
                ↪ e.flow >= lim){
                    adj_cur[u].pb(i);
                }
            }
        } return 0;
    }
    T dfs(short u, T flow, vector<short> &S, bool save = 0){
        if(save) S.pb(u);
        if(u == t) return flow;
        for(; ptr[u] < sz(adj_cur[u]); ++ptr[u]){
            edge &e = adj[u][adj_cur[u][ptr[u]]];
            if(T pushed = dfs(e.to, min(flow, e.cap - e.flow),
            ↪ S, save)){
                e.flow += pushed;
                adj[e.to][e.rev].flow -= pushed;
                if(e.cap-e.flow < lim) ptr[u]++;
                return pushed;
            }
        } return 0;
    }
    ll get_max_flow(short source, short sink){
        s = source; t = sink;
        mysort();
        vector<short> S;
        ll flow = 0;
        lim = SCALING ? (1<<30) : 1;
        for(; 0 < lim; lim >>= 1){
            while(bfs()){
                memset(ptr, 0, sizeof(ptr));
                while(T pushed = dfs(s, INF, S))
                    flow += pushed;
            }
        } return flow;
    }
    vector<short> get_st_cut(){
        vector<short> S;
        memset(ptr, 0, sizeof(ptr));
        dfs(s, INF, S, true);
        return S;
    }
};
```

MCMF. Complejidad: $O(\text{mín}\{|E|^2|V|\log|V|,\ F|E|\log|V|\})$.

```
const int MAXV = (1 << 15) - 1;
template<class T = ll> struct mcmf{
    mcmf(short V){this->V = V;}
    short s, t, V;
    const T INF = numeric_limits<T>::max();
    vector<T> p;
    struct edge{
        short to, rev;
        T cap, flow, mcap, w;
        bool operator<(const edge &b)const{return mcap >
        ↪ b.mcap;}
    };
    vector<edge> adj[MAXV];
    void add_edge(short u, short v, T cap, T w, bool
    ↪ is_directed = 1){
        if(u == v) return;
        T add = (is_directed ? 0 : cap);
        adj[u].pb({v, sz(adj[v]), cap, 0, cap + add, w});
        adj[v].pb({u, sz(adj[u]) - 1, add, 0, cap + add, -w});
    }
    struct pos{
        short from;
        T c;
        const bool operator<(const pos &b)const{return c > b.c;}
    };
    vector<T> bellman_ford(){
        vector<T> d(V);
        for(int i = 0; i < V - 1; ++i){
            for(int u = 0; u < V; ++u){
                for(edge &e : adj[u]){
                    if(d[e.to] > d[u] + e.w && e.cap > 0){
                        d[e.to] = d[u] + e.w;
                    }
                }
            }
        } return d;
    }
    pair<T, T> dijkstra(const T MAX_FLOW){
        vector<T> d(V, INF);
        vi P(V, -1), P_e(V, -1);
        priority_queue<pos> q;
        q.push({s, 0});
        d[s] = 0;
        while(sz(q)){
            pos act = q.top(); q.pop();
            if(act.c != d[act.from]) continue;
            for(int j=0; j<sz(adj[act.from]); ++j){
                edge &e = adj[act.from][j];
                if(e.cap - e.flow <= 0) continue;
                T _w = e.w+p[act.from] - p[e.to];
                if(d[e.to] <= d[act.from] + _w)
                    continue;
                d[e.to] = d[act.from] + _w;
                q.push(pos{e.to, d[e.to]});
                P[e.to] = act.from;
                P_e[e.to] = j;
            }
        }
        for(int i = 0; i < V; ++i) if(d[i] < INF)
            d[i] += -p[s] + p[i];
        for(int i = 0; i < V; ++i) if(d[i] < INF)
            p[i] = d[i];
        if(P[t] == -1) return {0, 0};
        T flow = MAX_FLOW;
        int cur_node = t;
        while(cur_node != s){
            flow = min(flow, adj[ P[cur_node] ][ P_e[cur_node]
            ↪ ].cap - adj[ P[cur_node] ][ P_e[cur_node]
            ↪ ].flow);
            cur_node = P[cur_node];
        }
```

```
        T new_cost = 0;
        cur_node = t;
        while(cur_node != s){
            adj[ P[cur_node] ][ P_e[cur_node] ].flow += flow;
            new_cost += adj[ P[cur_node] ][ P_e[cur_node] ].w
            ↪  * flow;
            adj[cur_node][adj[ P[cur_node] ][ P_e[cur_node]
            ↪  ].rev ].flow -= flow;
            cur_node = P[cur_node];
        }
        return {flow, new_cost};
    }
    pair<T, T> get_max_flow(short source, short sink, const T
    ↪  MAX_FLOW = 1e8){
        s = source;
        t = sink;
        p = bellman_ford();
        T flow = 0, cost = 0;
        while(flow < MAX_FLOW){
            pair<T, T> pushed = dijkstra(MAX_FLOW - flow);
            if(!pushed.fi) break;
            flow += pushed.fi;
            cost += pushed.se;
        }
        return {flow, cost};
    }
};
```

▶ SCC

Kosajaru Complejidad: Tiempo $O(n)$.

```
void dfs(int u, vi graph[], bool vis[], vi &topo_ord){
    if(vis[u]) return;
    vis[u] = true;
    for(int v : graph[u])dfs(v, graph, vis, topo_ord);
    topo_ord.pb(u);
}
void assign_scc(int u, vi inv_graph[], bool vis[], vi &scc,
↪  const int id){
    if(vis[u]) return;
    vis[u] = true;
    scc[u] = id;
    for(int v : inv_graph[u])
        assign_scc(v, inv_graph, vis, scc, id);
}
pair<int, vi> kosajaru(int n, vi graph[], vi inv_graph[]){
    bool vis[n] = {};
    vi topo_ord;
    for(int i=0; i<n; ++i) dfs(i,graph,vis,topo_ord);
    reverse(all(topo_ord));
    memset(vis, 0, sizeof(vis));
    vi scc(n);
    int id = 0;
    for(int u : topo_ord) if(!vis[u])
        assign_scc(u, inv_graph, vis, scc, id++);
    return {id, scc};
}
pair<vvi, vvi> build_scc_graph(int n, vi graph[], int max_scc,
↪  const vi &scc){
    vvi scc_graph(max_scc), inv_scc_graph(max_scc);
    for(int u = 0; u < n; ++u)
    for(int v : graph[u])
    if(scc[u] != scc[v])
        scc_graph[scc[u]].pb(scc[v]);
    for(int u = 0; u < n_scc; ++u){
        sort(all(scc_graph[u]));
        auto it = unique(all(scc_graph[u]));
        scc_graph[u].resize(it - scc_graph[u].begin());
        for(int v : scc_graph[u])
            inv_scc_graph[v].pb(u);
    } return {scc_graph, inv_scc_graph};
}
```

▶ 2-Sat

Complejidad: Tiempo en responder $O(n)$.

```
struct two_sat{
    int n;
    vvi graph, inv_graph;
    vi scc, ans;
    vector<bool> vis;
    two_sat(){}
    two_sat(int _n){
        n = _n;
        graph.resize(2 * n);
        inv_graph.resize(2 * n);
        scc.resize(2 * n);
        vis.resize(2 * n);
        ans.resize(n);
    }
    void add_edge(int u, int v){
        graph[u].pb(v);
        inv_graph[v].pb(u);
    }
    /// al menos una es verdadera
    void add_or(int p, bool val_p, int q, bool val_q){
        add_edge(p+(val_p? n:0), q+(val_q? 0:n));
        add_edge(q+(val_q? n:0), p+(val_p? 0:n));
    }
    /// exactamente una es verdadera
    void add_xor(int p, bool val_p, int q, bool val_q){
        add_or(p, val_p, q, val_q);
        add_or(p, !val_p, q, !val_q);
    }
    /// p y q tienen el mismo valor
    void add_and(int p, bool val_p, int q, bool val_q){
        add_xor(p, !val_p, q, val_q);
    }
    /// Kosajaru
    void dfs(int node, vi &topo_ord){...}
    void assign_scc(int node, const int id){...}
    /// construye respuesta
    bool build_ans(){
        fill(all(vis), false);
        vi topo_ord;
        for(int i=0; i<2*n; ++i)dfs(i, topo_ord);
        fill(all(vis), false);
        reverse(all(topo_ord));
        int id = 0;
        for(int u : topo_ord) if(!vis[u])
            assign_scc(u, id++);
        for(int i = 0; i < n; ++i){
            if(scc[i] == scc[i + n]) return 0;
            ans[i] = (scc[i]<scc[i + n] ? 0:1);
        } return 1;
    }
};
```

TREAP

T: $O(\log n)$ | **M:** $O(n)$ | *Para treap implicito eliminar las frecuencias freq.*

treap.cpp

```
struct _node{
    ll x, p;
    int freq, cnt, rev;
    _node *l, *r;
    _node() : l(nullptr), r(nullptr){}
    _node(ll _x): x(_x), p(seed()), freq(1), cnt(1), rev(0),
        l(nullptr), r(nullptr){}
    void recalc(){
        cnt = freq;
        cnt += ((l) ? l->cnt : 0) + ((r) ? r->cnt : 0);
    }
};
using node = _node*;
_node pool[MAXN + 2];
int nxt_node = 1;
node alloc(ll _x){
    pool[nxt_node] = _node(_x);
    return &pool[nxt_node++];
}
struct treap{
    node root;
    treap(): root(nullptr){}
    node merge(node l, node r){
        if(!l || !r) return l ? l : r;
        if(l->p < r->p){
            r->l = merge(l, r->l);
            r->recalc();
            return r;
        } else {
            l->r = merge(l->r, r);
            l->recalc();
            return l;
        }
    }
    pair<node, node> split_v(node u, ll d){
        if(!u) return {u, u};
        if(u->x < d){
            auto p = split_v(u->r, d);
            u->r = p.fi;
            u->recalc();
            return {u, p.se};
        } else {
            auto p = split_v(u->l, d);
            u->l = p.se;
            u->recalc();
            return {p.fi, u};
        }
    }
    void insert_v(ll x){
        node l, m, r;
        tie(l, r) = split_v(root, x);
        tie(m, r) = split_v(r, x + 1);
        if(m){
            m->freq++;
            m->cnt++;
        } else m = alloc(x);
        root = merge(merge(l, m), r);
    }
    void erase_v(ll x){
        node l, m, r;
        tie(l, r) = split_v(root, x);
        tie(m, r) = split_v(r, x + 1);
        if(!m || m->freq == 1) m = nullptr;
        else {
            m->freq--;
            m->cnt--;
        }
        root = merge(merge(l, m), r);
    }
};
struct implicit_treap{
    node root;
    implicit_treap(): root(nullptr){}
    void apply_lz(node u){ u->rev = 0; swap(u->l, u->r); }
    void push(node u){
        if(u && u->rev){
            apply_lz(u);
            if(u->l) u->l->rev ^= 1;
            if(u->r) u->r->rev ^= 1;
        }
    }
    node merge(node l, node r){
        push(l);
        push(r);
        if(!l || !r) return l ? l : r;
        if(l->p < r->p){
            r->l = merge(l, r->l);
            r->recalc();
            return r;
        } else {
            l->r = merge(l->r, r);
            l->recalc();
            return l;
        }
    }
    pair<node, node> split_p(node u, int pos){
        if(!u) return {u, u};
        push(u);
        int cur_pos = (u->l) ? (u->l->cnt) : 0;
        if(cur_pos < pos){
            auto p = split_p(u->r, pos - cur_pos - 1);
```

```
            u->r = p.fi;
            u->recalc();
            return {u, p.se};
        } else {
            auto p = split_p(u->l, pos);
            u->l = p.se;
            u->recalc();
            return {p.fi, u};
        }
    }
    void insert_p(int pos, ll x){
        node l, r;
        tie(l, r) = split_p(root, pos);
        root = merge(merge(l, alloc(x)), r);
    }
    void erase_p(int pos){
        node l, m, r;
        tie(l, r) = split_p(root, pos);
        tie(m, r) = split_p(r, 1);
        m = nullptr;
        root = merge(l, r);
    }
    void reverse(int l, int r){
        node prv, inter, nxt;
        tie(prv, inter) = split_p(root, l);
        tie(inter, nxt) = split_p(inter, r - l + 1);
        inter->rev ^= 1;
        root = merge(merge(prv, inter), nxt);
    }
    void _print(node u){
        if(!u) return;
        push(u);
        _print(u->l);
        cout << u->x << ' ';
        _print(u->r);
    }
    void print(){ _print(root); }
};
```

STRINGS

▸ Hashing

T: $O(|s|)$ | **M:** $O(|s|)$ | *Calcula el hash de los prefijos de un string. Admite concatenacion y potencias.*

hashing.cpp

```
ll bin_exp( ll a, ll exp, ll mod);
ll inv( ll b , ll mod);
ll div( ll a, ll b, ll mod);
struct mystring{
    const ll A = 911382323;
    const ll B = 972663749;
    string s;
    ll hash_val = -1, len = 0;
    mystring(string &s): s(s){
        len = sz(s);
        hash_precalc();
    }
    mystring(ll hash_val, ll len):hash_val(hash_val), len(len){}
    bool operator==(const mystring &o)const{
        return (hash_val == o.hash_val) && (len == o.len);
    }
    bool operator!=(const mystring &other)const{
        return !(*this == other);
    }
    vi h, powA;
    ll hash_precalc(){
        if(hash_val != -1) return hash_val;
        if(len == 0) return hash_val = 0;
        h.resize(len);
        powA.resize(len + 1);
        h[0] = s[0] % B;
        powA[0] = 1;
        for(int i = 1; i < sz(s); ++i){
            h[i] = (h[i-1] * A + s[i]) % B;
            powA[i] = A * powA[i-1] % B;
        }
        powA[len] = powA[len - 1] * A % B;
        hash_val = h[sz(s) - 1];
        return hash_val;
    }
    mystring hash_substring(ll i, ll j){
        if(hash_val == -1) hash_precalc();
        if(i < 0 || j >= len || i > j) return mystring(0, 0);
        if(i == 0) return mystring(h[j], j + 1);
        ll nhash = (h[j] - h[i-1] * powA[j - i + 1] % B + B)%B;
        return mystring(nhash, j - i + 1);
    }
    mystring operator*(ll p)const{
        if( p <= 0 || len == 0 ) return mystring(0, 0);
        if( p == 1) return mystring(hash_val, len);
        ll mult = bin_exp(A, len,   B);
        ll sum_geo;
        if(mult == 1) sum_geo = p % B;
        else{
            ll num = (bin_exp(mult, p,  B) - 1 + B) % B;
            sum_geo = div(num, (mult - 1 + B) % B, B);
        }
        ll new_hash = hash_val * sum_geo % B;
        ll new_len = len * p;
        return mystring(new_hash, new_len);
    }
    mystring operator+(const mystring &o)const{
        ll pA = bin_exp(A, o.len, B);
        ll new_hash = (hash_val * pA % B + o.hash_val) % B;
        ll new_len = len + o.len;
        return mystring(new_hash, new_len);
    }
};
```

▸ KMP

Complejidad: Tiempo $O(|s|)$ - Memoria extra $O(|s|)$.

```
vi prefix_function(const string &s){
    int n = sz(s);
    vi pi(n);
    for(int i = 1; i < n; ++i){
        int j = pi[i - 1];
        while(j && s[i] != s[j]) j = pi[j - 1];
        pi[i] = j + (s[i] == s[j]);
    } return pi;
}
```

Autómata de KMP. Complejidad: Tiempo $O(|s|k)$ - Memoria extra $O(|s|k)$, donde k es el tamaño del alfabeto.

```
void compute_automaton(const string &s, vvi& aut){
    s += '#';
    int n = sz(s);
    vi pi = prefix_function(s);
    aut.assign(n, vi(26));
    for(int i = 0; i < n; ++i){
        for(int c = 0; c < 26; ++c){
            if(i && 'a' + c != s[i])
                aut[i][c] = aut[pi[i - 1]][c];
            else aut[i][c] = i + ('a'+c == s[i]);
        }
    }
}
```

▸ Suffix Automata

Complejidad: Tiempo $O(|s|)$, memoria $O(|S|)$. Construye un autómata sobre los sufijos de s, donde cada nodo terminal es un sufijo de s. Esta construcción se hace online. Donde *alpha* es el tamaño del alfabeto y $offset$ el primer caracter. Si se hace uso de los nodos terminales llamar a markTerminalNodes() primero (este método no es online).

```
constexpr short alpha = 26;
constexpr char offset = 'a';
struct state{
    //max(state) = len, min(state) = sa[link].len
    int len, link;
    bool is_terminal;
    array<int,alpha> next;
    state(){
        len = 0; link = -1; is_terminal = false;
        next.fill(0);
    }
};
struct SuffixAutomaton{
    int n;
    vector<state> sa;
    int sz = 1, last = 0;
    SuffixAutomaton(int n) : n(n), sa(2*n + 1){}
    void add(char ch_){
        short ch = ch_-offset;
        int curr = sz++;
        sa[curr].len = sa[last].len + 1;
        int p = last;
        while(p!=-1 && !sa[p].next[ch]){
            sa[p].next[ch] = curr;
            p = sa[p].link;
        }
        if(p == -1){
            sa[curr].link = 0;
        }else{
            int q = sa[p].next[ch];
            if(sa[p].len+1 == sa[q].len){
                sa[curr].link = q;
            }else{
                int clone = sz++;
                sa[clone].len = sa[p].len + 1;
                sa[clone].next = sa[q].next;
                sa[clone].link = sa[q].link;
                while(p!=-1 && sa[p].next[ch] == q){
                    sa[p].next[ch] = clone;
                    p = sa[p].link;
                }
                sa[q].link = sa[curr].link = clone;
            }
        }
        last = curr;
        return;
    }
    void markTerminalNodes(){
        int curr = last;
        while(curr){
            sa[curr].is_terminal = true;
            curr = sa[curr].link;
        }
        return;
    }
};
```

▸ Suffix array

Construcción. Complejidad: Tiempo $O(|s|\log(|s|))$ - Memoria $O(|s|)$. Calcula la permutación que corresponde a los sufijos ordenados lexicográficamente. `SA[i]` es el índice en el cual empieza el i-ésimo sufijo ordenado.

```
int SA[MAXN], mrank[MAXN];
int tmpSA[MAXN], tmp_mrank[MAXN];
void counting_sort(int k, int n){
    int freqs[MAXN] = {};
    for(int i = 0; i < n; ++i){
        if(i + k < n) freqs[ mrank[i + k] ]++;
        else freqs[0]++;
    }
    int m = max(100, n);
    for(int i = 0, sfs = 0; i < m; ++i){
        int f = freqs[i];
        freqs[i] = sfs;
        sfs += f;
    }
    for(int i = 0; i < n; ++i){
        if(SA[i] + k < n) tmpSA[ freqs[mrank[ SA[i] + k ]]++ ]
        ↪  = SA[i];
        else tmpSA[ freqs[0]++ ] = SA[i];
```

```
    }
    for(int i = 0; i < n; ++i) SA[i] = tmpSA[i];
}
void buildSA(string &str){
    int n = sz(str);
    for(int i = 0; i < n; ++i){
        mrank[i] = str[i] - '#';
        SA[i] = i;
    }
    for(int k = 1; k < n; k <<= 1){
        counting_sort(k, n);
        counting_sort(0, n);
        int r = 0;
        tmp_mrank[ SA[0] ] = 0;
        for(int i = 1; i < n; ++i){
            if(mrank[ SA[i] ] != mrank[ SA[i - 1] ] || mrank[
            ↪  SA[i] + k ] != mrank[ SA[i - 1] + k ])
                tmp_mrank[ SA[i] ] = ++r;
            else tmp_mrank[ SA[i] ] = r;
        }
        for(int i = 0; i < n; ++i) mrank[i] = tmp_mrank[i];
    }
}
inline bool suff_compare1(int idx,const string &pattern) {
    return (s.substr(idx).compare(0, sz(pattern), pattern) < 0);
}
inline bool suff_compare2(const string &pattern,int idx) {
    return (s.substr(idx).compare(0, sz(pattern), pattern) > 0);
}
pair<int,int> match(const string &pattern) {
    int *low = lower_bound (SA, SA + sz(s), pattern,
    ↪  suff_compare1);
    int *up = upper_bound (SA, SA + sz(s), pattern,
    ↪  suff_compare2);
    return make_pair((int)(low - SA),(int)(up - SA));
}
```

Prefijo común más largo. Complejidad: Tiempo $O(|s|)$ - Memoria $O(|s|)$. Calcula la longitud del prefijo común más largo entre dos sufijos consecutivos (lexicográficamente) de s. *lcp*[*i*] guarda la respuesta para el i-ésimo sufijo y el $(i-1)$-ésimo sufijo.

```
int lcp[MAXN];
void build_lcp(string &str){
    int n = sz(str);
    int phi[n];
    phi[SA[0]] = -1;
    for(int i = 1; i < n; ++i) phi[ SA[i] ] = SA[i - 1];
    int plcp[n];
    int k = 0;
    for(int i = 0; i < n; ++i){
        if(phi[i] == -1){
            plcp[i] = 0;
            continue;
        }
        while(i + k < n && phi[i] + k < n && str[i + k] ==
        ↪  str[phi[i] + k]) k++;
        plcp[i] = k;
        k = max(k - 1, 0);
    }
    for(int i = 0; i < n; ++i) lcp[i] = plcp[SA[i]];
}
```

▶ Aho-Corasick

Construción en $O(mk)$, donde m es el tamaño total de los strings y k el tamaño del alfabeto.

```
struct aho_corasick{
    const static int K = 26;
    const char index = 'a';
    struct vertex{
        int next[K];
        bool terminal = false;
        int p = -1;
        char p_edge;
        int link = -1;
        int terminal_link = -1;
        int go[K];
        vertex(int p = -1, char c = '$') : p(p), p_edge(c){
            memset(next, -1, K*sizeof(int));
            memset(go, -1, K*sizeof(int));
        }
    };
    vector<vertex> aho;
    aho_corasick(){ aho.resize(1); }
    void add_string(const string &s){
        int u = 0;
        for(char c : s){
            int e = c - index;
            if(aho[u].next[e] == -1){
                aho[u].next[e] = sz(aho);
                aho.emplace_back(u, c);
            }
            u = aho[u].next[e];
        } aho[u].terminal = 1;
    }
    int get_link(int u){
        if(aho[u].link == -1){
            if(!u || !aho[u].p) aho[u].link = 0;
            else aho[u].link = go(get_link(aho[u].p),
            ↪  aho[u].p_edge);
        } return aho[u].link;
    }
    int go(int u, char c){
        int e = c - index;
        if(aho[u].go[e] == -1){
            if(aho[u].next[e] != -1)
                aho[u].go[e] = aho[u].next[e];
            else aho[u].go[e] = !u ? 0 : go(get_link(u), c);
        } return aho[u].go[e];
    }
    int get_terminal_link(int u){
        if(aho[u].terminal_link == -1){
            if(!u || !aho[u].p)
                aho[u].terminal_link = 0;
            else aho[u].terminal_link =
            ↪  get_terminal_link(get_link(u));
        } return aho[u].terminal_link;
    }
};
```

▶ Suffix tree

```
/// MEJORAR ESTA COSA, SOLO LO COPIE Y
↪  PEGUÉ porcuestionesdetiempo
const int inf = 1e9;
const int maxn = 1e6;
int s[maxn];
map<int, int> to[maxn];
//Root is the vertex 0
//f_pos[i] is the initial index with the letter of the edge
↪  that goes from the parent of i to i
//len[i] is the number of letters in the edge that enters in i
//slink[i] is the suffix link
int len[maxn], f_pos[maxn], slink[maxn];
int node, pos;
int sz = 1, n = 0;
int make_node(int _pos, int _len){
    f_pos[sz] = _pos;
    len [sz] = _len;
    return sz++;
}
void go_edge(){
    while(pos > len[to[node][s[n - pos]]]){
        node = to[node][s[n - pos]];
        pos -= len[node];
    }
}
void add_letter(int c){
    s[n++] = c;
    pos++;
    int last = 0;
    while(pos > 0){
        go_edge();
        int edge = s[n - pos];
        int &v = to[node][edge];
        int t = s[f_pos[v] + pos - 1];
        if(v == 0){
            v = make_node(n - pos, inf);
            //v = make_node(n - pos, 1);
            slink[last] = node;
            last = 0;
        } else if(t == c) {
            slink[last] = node;
            return;
        } else {
            int u = make_node(f_pos[v], pos - 1);
            to[u][c] = make_node(n - 1, inf);
            to[u][t] = v;
            f_pos[v] += pos - 1;
            len [v] -= pos - 1;
            v = u;
            slink[last] = u;
            last = u;
        }
        if(node == 0) pos--;
        else node = slink[node];
    }
}
void correct(int s_size){
    len[0] = 0;
    for (int i = 1; i < sz; i++){
        if (f_pos[i] + len[i] - 1 >= s_size){
            len[i] = (s_size - f_pos[i]);
        }
    }
}
void print_suffix_tree(int from){
    cout << "Edge entering in " << from << " has size " <<
    ↪  len[from];
    cout << " and starts in " << f_pos[from] << endl;
    cout << "Node " << from << " goes to: ";
    for (auto u : to[from]){
        cout << u.se << " with " << (char)u.fi << " ";
    }
    cout << endl;
    for (auto u : to[from]){
        print_suffix_tree(u.se);
    }
}
void build(string &s){
    for (int i = 0; i < sz; i++){
        to[i].clear();
    }
    sz = 1;
    node = pos = n = 0;
    len[0] = inf;
    for(int i = 0; i < sz(s); i++)
        add_letter(s[i]);
    correct(sz(s));
}
void cutGeneralized(vi &finishPoints){
    for (int i = 0; i < sz; i++){
        int init = f_pos[i];
        int end = f_pos[i] + len[i] - 1;
        int idx = lower_bound(all(finishPoints), init) -
        ↪  finishPoints.begin();
        if ((idx != sz(finishPoints)) && (finishPoints[idx] <=
        ↪  end)){//Must be cut
            len[i] = (finishPoints[idx] - f_pos[i] + 1);
            to[i].clear();
        }
    }
}
void build_generalized(vector<string> &ss){
    for (int i = 0; i < sz; i++){
        to[i].clear();
    }
    sz = 1;
    node = pos = n = 0;
    len[0] = inf;
    int sep = 256;
```

```
    vi finishPoints;
    int next = 0;
    for (int i = 0; i < sz(ss); i++){
        for (int j = 0; j < sz(ss[i]); j++){
            add_letter(ss[i][j]);
        }
        next += sz(ss[i]);
        finishPoints.pb(next);
        add_letter(sep++);
        next++;
    }
    correct(next);
    cutGeneralized(finishPoints);
}
```

GEOMETRÍA

▸ Convex hull

Complejidad: $O(n \log n)$.

```
/// MEJORAR ESTA COSA, SOLO LO COPIE Y PEGUÉ porcuestionesdetiempo
struct pt {
    double x, y;
};
int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y) + c.x*(a.y-b.y);
    if(v < 0) return -1; // clockwise
    if(v > 0) return +1; // counter-clockwise
    return 0;
}
bool cw(pt a, pt b, pt c, bool include_collinear){
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && !o);
}
bool collinear(pt a, pt b, pt c){
    return orientation(a, b, c) == 0; }
void convex_hull(vector<pt>& a, bool include_collinear = 0){
    pt p0 = *min_element(all(a), [](pt a, pt b){
        return make_pair(a.y, a.x) < make_pair(b.y, b.x);
    });
    sort(all(a), [&p0](const pt& a, const pt& b){
        int o = orientation(p0, a, b);
        if(o == 0)
        return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
                < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
        return o < 0;
    });
    if(include_collinear){
        int i = sz(a)-1;
        while(i >= 0 && collinear(p0, a[i], a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }
    vector<pt> st;
    for(int i = 0; i < sz(a); i++){
        while(sz(st)>1 && !cw(st[sz(st)-2], st.back(), a[i],
        ↪  include_collinear))
            st.pop_back();
        st.pb(a[i]);
    }
    a = st;
}
```

TEORÍA DE JUEGOS

▸ Green Numbers

Dos jugadores alternan turnos, información completa, sin azar ni empates. *Imparcial* si ambos tienen los mismos movimientos desde todo estado; otro caso es *partidario*. *Normal* el último en mover gana, en *Misère*, pierde.

Grafo de estados. $u \to v$ representa un movimiento. Los terminales son perdedores (P); u es ganador (G) sii tiene una salida a P, y es P sii todas sus salidas van a G. Orden topológico inverso.

Caracterizar los P: propiedad $\mathcal{P}$: los terminales la cumplen; todo movimiento desde $\mathcal{P}$ la rompe; desde $\neg\mathcal{P}$ existe un movimiento a $\mathcal{P}$.

Nim y Grundy En Nim, un movimiento reduce una de las pilas $a_1, \ldots, a_k$. Sea $X = \bigoplus_i a_i$: la posición es P sii $X = 0$. Si $X \neq 0$, sea b su bit encendido más significativo; elegir a_i con b encendido y reemplazarla por $a_i \oplus X < a_i$ deja XOR 0.

Para una posición s, $G(s) = \text{mex}\{G(t) : s \to t\}$. Entonces s es P sii $G(s) = 0$. **Sprague–Grundy:** todo juego imparcial finito normal equivale a una pila de Nim de tamaño $G(s)$; para juegos independientes, $G_{\text{total}} = G(s_1) \oplus \cdots \oplus G(s_k)$, y el estado global es P sii $G_{\text{total}} = 0$.

Finales especiales: Si el problema no define los estados perdedores en términos de 'no tener movimientos' (por ejemplo, el objetivo es mover una ficha a una casilla determinada), debemos transformar el juego identificando los nodos que obligan al oponente dejarnos en una posición ganadora. Dichos nodos actuarán como nuestros estados terminales.

Si quien no puede mover gana, es **misère** y el XOR de Grundy **no aplica**.

Maximización/minimización. Si A maximiza F y B la minimiza eligiendo objetos alternadamente: analizar los dos objetos finales para ambos turnos, deducir la condición de intercambio, ordenar con ella y generalizar a n objetos.

▸ Green Hackenbush

Grafo unido al *suelo*; se elimina una arista y desaparece todo lo que pierda conexión con él. Es imparcial normal y las componentes se combinan con XOR.

Principio del colon. Si $G(H_1) = G(H_2)$ con H_1, H_2 arboles y H_1 se une al resto del grafo sólo por una articulación, sustituir H_1 por H_2 no cambia el valor del numero de Grundy de todo el grafo. Esto se reduce a $G(u) = \bigoplus_{v \text{ hijo de } u}(G(v) + 1)$, para arbol enraizado.

Principio de fusión. Fusionar todos los vértices de un ciclo preserva Grundy; sus aristas se vuelven lazos. Unificar el suelo, fusionar ciclos. Reemplazar cada lazo por una hoja y aplicar colon. Lazos en un mismo vértice se cancelan por pares.

T: $O((N+M)\alpha(N))$ | **M:** $O(N+M)$ | *Nodos en* $[0, N)$. *Grafo general con* M *aristas.*

GreenHackenbush.cpp

```
struct dsu{ ... } struct edge{ int from, to; };
struct ghb_r{
    int cntRepre, root, value;
    vi compactados, paridad, nim; vvi tree;
};
ghb_r ghb(int n, vector<edge> edges, const vi &suelo={} ){
    vi norm(n); vvi adj(n); iota( all(norm), 0 );
    for( int  u : suelo) norm[u] = 0;
    for( int  i = 0; i < sz(edges); i++ ){
        edge &e = edges[i];
        e.from = norm[e.from], e.to = norm[e.to];
        adj[e.from].pb(i);
        if(e.from != e.to) adj[e.to].pb(i);
    }
    vi pen,dep(n,-1),low(n),par(n, -1),paredg(n, -1),nex(n,0);
    dsu uf(n); pen.pb(0); dep[0] = low[0] = 0;
    while( sz(pen)  ){
        int u = pen.back();
        if( nex[u] < sz(adj[u])  ){
            int id = adj[u][nex[u]++];
            if(id == paredg[u]) continue;
            const edge &e = edges[id];
            int v = (e.from == u ? e.to : e.from);
            if( v == u ) continue;
            if( dep[v] == -1  ){
                par[v] = u, paredg[v] = id;
                dep[v] = low[v] = dep[u] + 1; pen.pb(v);
            }
            else if(dep[v] < dep[u])
                low[u] = min(low[u], dep[v]);
        }else{
            pen.pop_back();
            int p = par[u];
            if(p == -1) continue;
            low[p] = min(low[p], low[u]);
            if(low[u] <= dep[p]) uf.join(p, u, false);
        }
    }
    ghb_r r; vi id(n, -1); r.cntRepre = 0;
    for( int  u = 0; u < n; u++ ){
        if(dep[u] == -1) continue;
        int ro = uf.root(u);
        if(id[ro] == -1) id[ro] = r.cntRepre++;
    }
    r.compactados.assign(n, -1);
    for( int  u = 0; u < n; u++ ){
        int v = norm[u];
        if(dep[v] != -1) r.compactados[u] = id[uf.root(v)];
    }
    r.root = r.compactados[0];
    r.tree.resize(r.cntRepre);r.paridad.assign(r.cntRepre,0);
    for(const edge &e : edges ){
        if(dep[e.from] == -1) continue;
        int a = id[uf.root(e.from)], b = id[uf.root(e.to)];
        if(a == b) r.paridad[a] ^= 1;
        else{ r.tree[a].pb(b); r.tree[b].pb(a); }
    }
    for( int  u = 0; u < r.cntRepre; u++ ){
        if(!r.paridad[u]) continue;
        int leaf = sz(r.tree);
        r.tree.pb(vi{u});
        r.tree[u].pb(leaf);
    }
    vi order={r.root},npar(sz(r.tree),-1);npar[r.root]=r.root;
    for( int  i = 0; i < sz(order); i++ ){ int u = order[i];
        for( int  v : r.tree[u] ){
            if(v == npar[u]) continue;
            npar[v] = u; order.pb(v);
        }
    }
    r.nim.assign(sz(r.tree), 0);
    for( int  i = sz(order) - 1; i > 0; i-- ){
        int u = order[i]; r.nim[npar[u]] ^= (r.nim[u]+1);
    }
    r.value = r.nim[r.root]; return r;
}
```

T: $O(N+M)$ | **M:** $O(N+M)$ | *Nodos en* $[0, N)$. *Arbol con* M *multi-aristas.*

```
struct edge{int from, to;ll w;};
struct ghb_r{ int cntRepre, root=0, value; vi nim; vvpii tree;};
ghb_r ghb(int n, vector<edge> edges, const vi &suelo={} ){
    ghb_r r;r.cntRepre = n;r.root = 0;r.tree.resize(r.cntRepre);
    for(const edge &e : edges ){
        int a = e.from; int b = e.to; ll w = e.w;
        r.tree[a].pb({b,w}); r.tree[b].pb({a,w});
    }
    vpii order={ {r.root, 0} }; vi npar(sz(r.tree),-1);
    npar[r.root]=r.root;
    for( int  i = 0; i < sz(order); i++ ){ int u = order[i].fi;
        for( pii  arista : r.tree[u] ){
            int v = arista.fi; ll w = arista.se;
            if( v == npar[u] ) continue;
            npar[v] = u;order.pb({v,w}); // u -> v con  w veces
        }
    }
    r.nim.assign(sz(r.tree), 0);
    for( int  i = sz(order) - 1; i > 0; i-- ){
        int u = order[i].fi;  ll w = order[i].se;
```

```
        if( w == 1){ r.nim[npar[u]] ^= (r.nim[u]+ 1 );
        }else{ r.nim[npar[u]] ^= (r.nim[u]);
            int restantes = w%2; r.nim[npar[u]] ^= (restantes);
        }
    }
    r.value = r.nim[r.root]; return r;
}
```

DINAMIC PROGRAMMING

▸ Subset sum optimization.

Complejidad: $O(N\sqrt{N})$. Si queremos obtener calcular el conjunto de sumas posibles en subconjuntos de un arreglo A, en el cual se cumple que $\sum a_i = N$ y todo $a_i \geq 0$, podemos obtener exactamente el mismo conjunto con un arreglo comprimido de tamaño $\sqrt{N}$. El $3k-trick$ explota que las sumas generadas por el arreglo $[a, a, a]$ son exactamente las mismas que en $[2a, a]$. Podemos juntar elementos y repetirlo hasta que a solo haya dos copias de cada elemento distinto en el nuevo arreglo, acotando el numero de elementos a $O(\sqrt{N})$. El proceso de compresión toma $O(N \log_2(N))$.

```
vi compress(vector<ll>& a){
    sort(a.rbegin(), a.rend());
    int n = sz(a);
    a.pb(0);
    vi weights;
    int pi = 0;
    for(int i = 1; i <= n; i++){
      if(a[i] != a[i - 1]){
        ll cnt = i - pi;
        ll x = a[i - 1];
        ll j = 1;
        while(j < cnt){
          weights.pb(x * j);
          cnt -= j;
          j *= 2;
        }
        weights.pb(x * cnt);
        pi = i;
      }
    } return weights;
}
```

▸ Subset Sum con bitset

Dado una una arreglo de tamaño n ver si es psoible encontrar es la suma objetivo S. Nota: w es el tamaño del bloque de bits procesados en paralelo (normalmente 32 o 64). Complejidad: Temporal $O\left(n \times \frac{S}{w}\right)$ - Memoria $O(S)$

```
bool subsetSumBitset( vi& arr, int S) {
    bitset<10001> dp; // tam >= S+1
    dp[0] = 1;
    for(int x : arr) dp |= (dp << x);
    return dp[S];
}
```

▸ SOS DP

Dada $f(X)$ definida para cada $X \subseteq \{0, \ldots, n-1\}$, queremos calcular $g(X) = \sum_{T \subseteq X} f(T)$. Definimos $S(x, i) = \{y \subseteq x \mid y \oplus x < 2^i\}$.

Notar que los bits que pueden variar son los $\{0, 1, \ldots, i-1\}$ (si el bit i de x es 1), apartir del bit i ya está fijo. $S(x, 0) = \{x\}$ y $S(x, n) = \{\text{todos los submasks de x}\}$

Ejemplo 1: con $x = 11010110_2$ e $i = 3$:

$$S(x, 3) = \{11010000, 11010010, 11010100, 11010110\}$$

Ejemplo 2: con $x = 10110101_2$ e $i = 5$:

$$S(x, 5) = \begin{Bmatrix} 10100000, & 10100001, & 10100100, & 10100101, \\ 10110000, & 10110001, & 10110100, & 10110101 \end{Bmatrix}$$

Podemos descomponer $S(x, i)$ recursivamente:

$$S(x, i+1) = \begin{cases} S(x, i) & \text{si el bit } i \text{ de } x \text{ es } 0 \\ S(x, i) \sqcup S(x \oplus 2^i, i) & \text{si el bit } i \text{ de } x \text{ es } 1 \end{cases}$$

Definimos la tabla de DP $dp[x][i] = \sum_{y \in S(x,i)} f[y]$ y notamos que $g(x) = dp[x][n]$, $dp[x][0] = f(x)$ y la recurrencia es

$$dp[x][i+1] = \begin{cases} dp[x][i] & \text{si el bit } i \text{ de } x \text{ es } 0 \\ dp[x][i] + dp[x \oplus 2^i][i] & \text{si el bit } i \text{ de } x \text{ es } 1 \end{cases}$$

T: $O(n \cdot 2^n)$ | **M:** $O(n \cdot 2^n)$ |

```
int n; // leer n
vector<int> f(1<<n); // inicializar f[S]
vector<vector<int>> dp(1<<n, vector<int>(n + 1));
for(int x = 0; x < (1<<n); x++) dp[x][0] = f[x];
for(int i = 0; i < n; i++ )
    for(int x = 0; x < (1<<n); x++ ){
        if(x&(1<<i)) dp[x][i+1] = dp[x][i] + dp[x^(1<<i)][i];
        else         dp[x][i+1] = dp[x][i];
    }
```

T: $O(n \cdot 2^n)$ | **M:** $O(2^n)$ |

```
int n;
vector<int> f(1<<n); // inicializar f[S]
vector<int> g = f;
for(int i = 0; i < n; i++ )
    for(int mask = 0; mask < (1<<n); mask++ )
        if(mask & (1<<i)) g[mask] += g[mask^(1<<i)];
```

▸ Li-Chao Tree

La construcción inicial toma $\mathcal{O}(N \log N)$. Dado un conjunto $A \subset \mathbb{R}$ con $|A| = N$ y M funciones que cumplen la *transcending property*, la estructura permite agregar una nueva función en $\mathcal{O}(\log N)$ por operación. Además, para cualquier $a \in A$, permite responder Q consultas de la forma $\max_i f_i(a)$ en $\mathcal{O}(\log N)$ cada una. Complejidad total: $\mathcal{O}((N + M + Q) \log N)$. Notar que si hacemos insersiones por segmentos la complejidad es $O(\log^2 N)$.

Versión Extendida:

- ***Propiedad Transcendente (Transcending Property)***

El Li Chao Tree funciona para cualquier familia de funciones continuas $\{f_i\}$ definidas sobre el dominio de interés que cumplan la **propiedad de intersección única** o *transcending property*:

> *Para cualquier par de funciones distintas f_i y f_j, la función diferencia $d(x) = f_i(x) - f_j(x)$ cambia de signo a lo sumo una vez en el intervalo de evaluación $[x_{\text{mín}}, x_{\text{máx}}]$.*

Esto garantiza que, al comparar dos funciones en el extremo izquierdo `lv` y en el punto medio `mv`, la decisión sobre en qué mitad del intervalo se encuentra la intersección permanece binaria, permitiendo descartar una subrama en $\mathcal{O}(1)$ por nivel.

T: $O(\log N)$ | **M:** $O(N)$ |

li_chao_tree.cpp

```
struct Func {
  ll m,b;
  ll eval(ll x){
    if( m == LLONG_MAX) return LLONG_MAX;
    return (ll)((__int128_t)m * x + b);
  }
  Func(){ m = LLONG_MAX;}
  Func(ll m_, ll b_): m(m_), b(b_){ }
};
ostream& operator<<(ostream &os, const Func &f){
    return  os << f.m << "x+" << f.b ;
}
struct LiChaoTree {
  vll vals;
  ll maxV;
  Func *treefunc;
  LiChaoTree(vll &vals_){
    vals = vals_;
    sort(all(vals));
        vals.erase( std::unique( all(vals) ), vals.end() );
    treefunc = new Func[sz(vals) * 4 + 5];
    maxV = sz(vals);
  }
  void addFunction(Func f){ addFunction(f, 1, 0, maxV); }
  void addFunction(Func f, ll v, int l, int r){
    int m = l + (r - l) / 2;
        ll mv = vals[m];
        ll lv = vals[l];
        bool lef = f.eval(lv) < treefunc[v].eval(lv); // min
        bool mid = f.eval(mv) < treefunc[v].eval(mv); // min
        if(mid) swap(treefunc[v], f);
        if(r - l == 1) return;
        else if(lef != mid) addFunction(f, 2 * v, l, m);
        else addFunction(f, 2 * v + 1, m, r);
  }
    void addSegFunction( Func fi, int l, int r){//[l,r)->[i,j)
        l = lower_bound(all(vals), (ll)l) - vals.begin();
        r = lower_bound(all(vals), (ll)r) - vals.begin();
        if( l < r ) addSeg(fi,l,r, 1,0,maxV );
    }
    void addSeg(Func fi,int l,int r,int v,int left,int right){
        if( r <= left  || right <= l) return;
        if( l <= left  && right <=  r  ){
            addFunction( fi, v, left, right);
            return;
        }
        if( left +1 == right)  return;
        int m =  left + (right-left)/2;
        addSeg(fi, l, r, v * 2     , left, m);
        addSeg(fi, l, r, v * 2 + 1 , m, right);
    }
    ll get(ll x){ return get(x, 1, 0, maxV); }
  ll get(ll x, int v, int l, int r){
        ll cur = treefunc[v].eval(x);
        if(r - l == 1) return cur;
        int m = l + (r - l) / 2;
        ll mv = vals[m];
        if(x < mv) return min(cur, get(x, 2 * v, l, m)); //min
        else return min(cur, get(x, 2 * v + 1, m, r)); //min
  }
  ~LiChaoTree(){ delete[] treefunc; }
};
```

▸ Li Chao Tree Dinámico

Versión Extendida:

Opera sobre un dominio entero/continuo arbitrario $[L, R]$ (ej. $[-10^9, 10^9]$) sin necesidad de compresión previa de coordenadas.

Operaciones y Complejidades Temporales ($U = R - L + 1$):

add_line(line) : Inserta una línea en todo el dominio $[L, R]$. $\mathcal{O}(\log U)$.

add_segment(line, l, r) : Inserta una línea restringida al intervalo espacial $[l, r]$. $\mathcal{O}(\log^2 U)$ (descompone en $\mathcal{O}(\log U)$ nodos canónicos y desciende).

query(x) : Consulta el valor mínimo $y = mx + b$ en el punto x considerando todo el dominio global.$\mathcal{O}(\log U)$

query_segment(x, l, r) : Consulta el valor mínimo en la coordenada x, filtrando **únicamente** las líneas cuyos nodos de inserción intersectan o cubren el rango de consulta $[l, r]$. Retorna ∞ si no hay líneas válidas.$\mathcal{O}(\log U)$

Memoria : $\mathcal{O}(Q \log^2 U)$ en el peor caso de inserciones de segmentos, debido a la creación dinámica de nodos (`new node`) bajo demanda.

Consideraciones Prácticas : El factor $\log U$ es pequeño (≈ 30 niveles para 10^9). Evita el uso de compresión offline, pero introduce sobrecarga de memoria y punteros en comparación con la versión estática basada en arreglos contiguos.

```cpp
#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const ll inf = 2e18;

struct Line {
  ll m, c;
  ll eval(ll x) {
    return m * x + c;
  }
};
struct node {
  Line line;
  node* left = nullptr;
  node* right = nullptr;
  node(Line line) : line(line) {}
  void add_segment(Line nw, int l, int r, int L, int R) {
    if (l > r || r < L || l > R) return;
    int m = (l + 1 == r ? l : (l + r) / 2);
    if (l >= L and r <= R) {
      bool lef = nw.eval(l) < line.eval(l);
      bool mid = nw.eval(m) < line.eval(m);
      if (mid) swap(line, nw);
      if (l == r) return;
      if (lef != mid) {
        if (left == nullptr) left = new node(nw);
        else left -> add_segment(nw, l, m, L, R);
      }
      else {
        if (right == nullptr) right = new node(nw);
        else right -> add_segment(nw, m + 1, r, L, R);
      }
      return;
    }
    if (max(l, L) <= min(m, R)) {
      if (left == nullptr) left = new node({0, inf});
      left -> add_segment(nw, l, m, L, R);
    }
    if (max(m + 1, L) <= min(r, R)) {
      if (right == nullptr) right = new node ({0, inf});
      right -> add_segment(nw, m + 1, r, L, R);
    }
  }
  ll query_segment(ll x, int l, int r, int L, int R) {
    if (l > r || r < L || l > R) return inf;
    int m = (l + 1 == r ? l : (l + r) / 2);
    if (l >= L and r <= R) {
      ll ans = line.eval(x);
      if (l < r) {
        if (x <= m && left != nullptr) ans = min(ans, left ->
        ↪  query_segment(x, l, m, L, R));
        if (x > m && right != nullptr) ans = min(ans, right ->
        ↪  query_segment(x, m + 1, r, L, R));
      }
      return ans;
    }
    ll ans = inf;
    if (max(l, L) <= min(m, R)) {
      if (left == nullptr) left = new node({0, inf});
      ans = min(ans, left -> query_segment(x, l, m, L, R));
    }
    if (max(m + 1, L) <= min(r, R)) {
      if (right == nullptr) right = new node ({0, inf});
      ans = min(ans, right -> query_segment(x, m + 1, r, L, R));
    }
    return ans;
  }
};

struct LiChaoTree {
  int L, R;
  node* root;
  LiChaoTree() : L(numeric_limits<int>::min() / 2),
  ↪  R(numeric_limits<int>::max() / 2), root(nullptr) {}
  LiChaoTree(int L, int R) : L(L), R(R) {
    root = new node({0, inf});
  }
  void add_line(Line line) {
    root -> add_segment(line, L, R, L, R);
  }
  // y = mx + b: x in [l, r]
  void add_segment(Line line, int l, int r) {
    root -> add_segment(line, L, R, l, r);
  }
  ll query(ll x) {
    return root -> query_segment(x, L, R, L, R);
  }
  ll query_segment(ll x, int l, int r) {
    return root -> query_segment(x, l, r, L, R);
  }
};

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  LiChaoTree t = LiChaoTree((int)-1e9, (int) 1e9);
  int n, q; cin >> n >> q;
  for (int i = 0; i < n; i++) {
    ll l, r, a, b; cin >> l >> r >> a >> b;
    r--;
    t.add_segment({a, b}, l, r);
  }
  while (q--) {
    int ty; cin >> ty;
    if (ty == 0) {
      ll l, r, a, b; cin >> l >> r >> a >> b;
      r--;
      t.add_segment({a, b}, l, r);
    }
    else {
      ll x; cin >> x;
      ll ans = t.query(x);
      if (ans >= inf) cout << "INFINITY\n";
      else cout << ans << '\n';
    }
  }
  return 0;
}
```

Versión Extendida:

Casos de Uso Avanzados (query_segment)

1. **Problemas de Caminos en Árboles (Tree Paths / Euler Tour):**
   - *Escenario:* Se modifican o agregan líneas en caminos arbitrarios sobre un árbol y se requiere consultar el óptimo en un nodo u considerando únicamente las líneas asociadas a su subárbol o a una ruta específica.
   - *Mapeo:* Al aplanar el árbol mediante un *Euler Tour*, los subárboles y rangos jerárquicos se transforman en intervalos continuos $[l, r]$, permitiendo realizar `add_segment` y `query_segment`$(x, in[u], out[u])$.
2. **Algoritmos de Barrido en el Plano (Sweep-Line con Ventanas Temporales/Espaciales):**
   - *Escenario:* Rectas o segmentos que "nacen" en una coordenada y "mueren."[en] otra (activos solo en un intervalo $[l, r]$).
   - *Mapeo:* Al barrer el plano, se insertan segmentos con su respectivo intervalo de vigencia y se consulta el valor óptimo filtrando estrictamente por las entidades activas en el rango actual.
3. **Estructuras de Datos Persistentes o Versionadas (por IDs de Operación):**
   - *Escenario:* Consultar el resultado óptimo considerando únicamente las modificaciones realizadas dentro de un rango de tiempos o identificadores de eventos $[id_{min}, id_{max}]$.

MISCELÁNEOS

▶ XOR Basis

T: $O(d)$ | **M:** $O(d)$ |

```cpp
int basis[d];// basis[i]: v\inBase cuyo lsb es i
int sz;       // size actual de la base (= rango r)
void insertVector(int mask) {
    for( int i = 0; i < d; i++ ){
        if( !is_on(mask,i) )continue; // continue if
        ↪  lsb(mask)!=i
        if( !basis[i] ){ // no hay vector en la base con ese bit
            basis[i] = mask;
            ++sz;
            return;
        }
        mask ^= basis[i]; // si ya hay, quitarle ese bit a mask
    }
}
```

Teorema de Rango–Nulidad. Sea $\varphi : W \to V$ una transformación lineal con W de dimensión finita. Entonces $\dim W = \dim \ker \varphi + \dim \operatorname{im} \varphi$.
Primer Teorema de Isomorfismo Utilizando $W/\ker \varphi \cong \operatorname{im} \varphi$, los tamaños de los conjuntos satisfacen: $|W| = |\ker \varphi| \cdot |\operatorname{im} \varphi|$.
Cardinalidad de un Espacio Vectorial Finito Si V es un espacio vectorial de dimensión finita sobre un cuerpo finito K, la cardinalidad de V cumple que: $|V| = |K|^n$, si $V \cong K^n$.
Teorema de las Fibras Sea $\varphi : W \to V$ una transformación lineal entre espacios vectoriales. Para todo vector $v \in \operatorname{im} \varphi$, la fibra (preimagen) $\varphi^{-1}(v)$ es una clase lateral del núcleo, dada por:

$$\varphi^{-1}(v) = w_0 + \ker \varphi = \{w_0 + k \mid k \in \ker \varphi\}$$

donde $w_0 \in W$ es cualquier solución particular tal que $\varphi(w_0) = v$.
Como consecuencia, existe una biyección entre $\ker \varphi$ y cualquier fibra no vacía $\varphi^{-1}(v)$, por lo que la cardinalidad de cada fibra coincide exactamente con la del núcleo:

$$|\varphi^{-1}(v)| = |\ker \varphi| \quad \forall\, v \in \operatorname{im} \varphi.$$

Resultado Sean $v_1, \ldots, v_n \in \mathbb{F}_2^d$ y sea $V = \operatorname{span}(v_1, \ldots, v_n)$ con $r = \dim V$. Se quiere contar, para $x \in \mathbb{F}_2^d$, cuántos $S \subseteq [n]$ cumplen $\bigoplus_{i \in S} v_i = x$. (Si $x \notin V$, la respuesta es 0.)

$$\#\left\{ S \subseteq [n] : \bigoplus_{i \in S} v_i = x \right\} = \begin{cases} 2^{n-r} & \text{si } x \in V, \\ 0 & \text{si } x \notin V. \end{cases}$$

UTILIDADES

▶ Bitsets de tamaño (casi) dinámico

El código ejecutado dentro de func(sz) usará un bitset con tamaño de la potencia de 2 más cercano a sz.

```cpp
template<int len = 1> void func(int sz){
    if(sz > len){
      func<std::min(len * 2, MAXLEN)>(sz);
      return;
    }
    bitset<len> bs;
    //usa aquí tu bitset dinámico
}
```

▶ Bitwise ops

```cpp
#define is_on(S, j) (S & (1ll << (j)))
#define set_bit(S, j) (S |= (1ll << (j)))
#define clear_bit(S, j) (S &= ~(1ll << (j)))
#define toggle_bit(S, j) (S ^= (1ll << (j)))
#define lsb(S) ((S) & -(S))
#define clear_lsb(S) (S &= (S - 1))
#define set_all(S, n) (S = (1ll << (n)) - 1ll)
#define clear_trailing_ones(S) (S &= (S + 1))
#define set_last_bit_off(S) (S |= (S + 1))
```

```
#define is_power_of_two(S) (!((S) & ((S) - 1)))
#define nearest_power_of_two(S) ((int)pow(2,
  ↪ (int)((log((double)(S)) / log(2)) + 0.5)) )
#define is_divisible_by_power_of_two(n, k) !((n) & ((1ll <<
  ↪ (k)) - 1))
#define modulo(S, N) ((S) & ((N) - 1)) // S%N, N=2^k
```

► Bitwise - builtin

```
// one plus the index of the least significant 1-bit of x, or
  ↪ if x is zero, returns zero.
int __builtin_ffs(int x)
// number of leading 0-bits in x, starting at the most
  ↪ significant bit position. If x is 0 is undefined
int __builtin_clz(unsigned int x)
// number of trailing 0-bits in x, starting at the least
  ↪ significant bit position. If x is 0  undefined.
int __builtin_ctz(unsigned int x)
// he parity of x, i.e. the number of 1-bits in x modulo 2.
int __builtin_parity(unsigned int x)
```

► Iterar

Dada una máscara m, iterar sobre todos sus subconjuntos desde el más grande hasta el más pequeño (ojo el código no itera sobre $x = m$). La complejidad de iterar sobre todas las submáscaras de todos los números de 1 a 2^nes $O(3^n)$.

```
void submasks(int mask){
    for( int x = mask; x;){
        --x &= mask;
    }
}
```

► Gospers' Hack

Sirve para generar todos las máscaras de n bits, que tengan exactamente k bits a 1 (y que sean menores o iguales que 2^n).

Complejidad $O\left(\binom{n}{k}\right)$?

```
void GospersHack(int n, int k) {
    int mask = (1 << k) - 1;
    int limit = (1 << n);
    while(mask < limit){
        int c = mask & - mask;
        int r = mask + c;
        mask = (((r ^ mask) >> 2) / c) | r;
    }
}
```

FORMULAZAS

► Lógica - Conjuntos - Bitwise

$\oplus$ es el xor (o diferencia simétrica de conjuntos).

1. $a|b = a \oplus b + a\&b$
2. $a \oplus (a\&b) = (a|b) \oplus b$
3. $b \oplus (a\&b) = (a|b) \oplus a$
4. $(a\&b) \oplus (a|b) = a \oplus b$
5. $a + b = a|b + a\&b = a \oplus b + 2(a\&b)$
6. $a - b = (a \oplus (a\&b)) - ((a|b) \oplus a) = ((a|b) \oplus b) - ((a|b) \oplus a)$
7. $a - b = (a \oplus (a\&b)) - (b \oplus (a\&b)) = ((a|b) \oplus b) - (b \oplus (a\&b))$

Si p, q son predicados, entonces:

1. $(p \vee q) \iff -(-p \wedge -q)$.
2. $(p \implies q) \iff (-p \vee q)$.
3. $(p \oplus q) \iff -(p \iff q)$.

► Teoria de numeros

Expontente de la potencia mas grande de p primo que divide a $n!$

$$\nu_p(n!) = \sum_{i=1}^{\lfloor \log_p n \rfloor} \left\lfloor \frac{n}{p^i} \right\rfloor = \frac{n - S_p(n)}{p-1},$$

Donde $S_p(n)$ es la suma de dígitos de n en base p.

► Combinatoria

- ***Números de Stirling del primer tipo***

Cantidad de permutaciones de n elementos con exactamente k ciclos disjuntos.

$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}, \quad \text{con } \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1, \begin{bmatrix} n \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ k \end{bmatrix} = 0$$

Función generatriz: $\sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k = x(x+1)\cdots(x+n-1)$.

$n\backslash k$	1	2	3	4	5	6	7	8
1	1							
2	1	1						
3	2	3	1					
4	6	11	6	1				
5	24	50	35	10	1			
6	120	274	225	85	15	1		
7	720	1764	1624	735	175	21	1	
8	5040	13068	13132	6769	1960	322	28	1
9	40320	109584	118124	67284	22449	4536	546	36

- ***Números de Stirling del segundo tipo***

Número de formas de particionar un conjunto de n elementos etiquetados en k subconjuntos no vacíos no etiquetados.

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = k\begin{Bmatrix} n-1 \\ k \end{Bmatrix} + \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix}, \quad \text{con } \begin{Bmatrix} 0 \\ 0 \end{Bmatrix} = 1, \begin{Bmatrix} n \\ 0 \end{Bmatrix} = \begin{Bmatrix} 0 \\ k \end{Bmatrix} = 0$$

T: $O(nk)$ | **M:** $O(nk)$ | *DP clásica mediante recurrencia*

Fórmula explícita / Convolución (Fila completa n):

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n = \sum_{j=0}^{k}\frac{(-1)^{k-j}}{(k-j)!}\cdot\frac{j^n}{j!}$$

T: $O(k \log k)$ | **M:** $O(k)$ | *Multiplicación de polinomios* $A_j = \frac{(-1)^j}{j!}$ *y* $B_j = \frac{j^n}{j!}$ *usando NTT.*

Cambio de base (Potencias a factoriales descendentes):

$$x^n = \sum_{k=0}^{n}\begin{Bmatrix} n \\ k \end{Bmatrix}(x)_k \quad \text{donde } (x)_k = x(x-1)\cdots(x-k+1)$$

$n\backslash k$	1	2	3	4	5	6	7	8	9
1	1								
2	1	1							
3	1	3	1						
4	1	7	6	1					
5	1	15	25	10	1				
6	1	31	90	65	15	1			
7	1	63	301	350	140	21	1		
8	1	127	966	1701	1050	266	28	1	
9	1	255	3025	7770	6951	2646	462	36	1
10	1	511	9330	34105	42525	22827	5880	750	45

- ***Números de Bernoulli***

Secuencia de números racionales B_m definidos por la función generatriz exponencial:

$$\frac{x}{e^x - 1} = \sum_{m=0}^{\infty} B_m \frac{x^m}{m!}$$

m	0	1	2	3	4	5	6	7	8	9	10
B_m	1	$-\frac{1}{2}$	$\frac{1}{6}$	0	$-\frac{1}{30}$	0	$\frac{1}{42}$	0	$-\frac{1}{30}$	0	$\frac{5}{66}$

Relación de Recurrencia:

$$\sum_{j=0}^{m}\binom{m+1}{j}B_j = 0 \implies B_m = -\frac{1}{m+1}\sum_{j=0}^{m-1}\binom{m+1}{j}B_j \quad (B_0 = 1)$$

T: $O(k^2)$ | **M:** $O(k)$ | *Cálculo directo por programación dinámica.*

Cálculo mediante Serie Formal de Potencias (Inversión): Dado que $e^x - 1 = \sum_{i=1}^{\infty}\frac{x^i}{i!}$, se invierte el polinomio $P(x) = \frac{e^x-1}{x} = \sum_{i=0}^{\infty}\frac{x^i}{(i+1)!}$.

$$B(x) = \sum_{m=0}^{\infty} B_m\frac{x^m}{m!} = \left(\sum_{i=0}^{\infty}\frac{x^i}{(i+1)!}\right)^{-1} \quad (\text{mód } x^{k+1})$$

T: $O(k \log k)$ | **M:** $O(k)$ | *Inversión de series con Newton / NTT.*

- ***Interpolación de Lagrange***

Dado un conjunto de $n+1$ puntos $\{(x_i, y_i)\}_{i=0}^{n}$ de un polinomio $P_n(x)$ de grado $\leq n$, el valor de $P_n(X)$ viene dado por:

$$P_n(X) = \sum_{i=0}^{n} y_i \ell_i(X), \quad \text{donde } \ell_i(X) = \prod_{\substack{j=0 \\ j\neq i}}^{n}\frac{X - x_j}{x_i - x_j}$$

T: $O(N^2)$ | **M:** $O(N)$ | *Para una única consulta* $P_n(x)$

Interpolación baricéntrica La fórmula baricéntrica de interpolación de Lagrange para el conjunto de nodos $\{(x_i, y_i)\}_{i=0}^{n}$ se define en su primera forma como:

$$P_n(x) = L(x)\sum_{i=0}^{n}\frac{w_i y_i}{x - x_i}, \quad \text{donde} \quad w_i = \left(\prod_{\substack{j=0 \\ j\neq i}}^{n}(x_i - x_j)\right)^{-1}$$

con el polinomio nodal:

$$L(x) = \prod_{k=0}^{n}(x - x_k) = \left(\sum_{i=0}^{n}\frac{w_i}{x - x_i}\right)^{-1}$$

Segunda forma baricéntrica:

$$P_n(x) = \left(\sum_{i=0}^{n}\frac{w_i y_i}{x - x_i}\right) \Big/ \left(\sum_{i=0}^{n}\frac{w_i}{x - x_i}\right)$$

T: $O(N^2)$ | **M:** $O(N)$ | *Cal* w_i *en* $O(N^2)$ *y consulta* $P_n(x)$ *en* $O(N)$

Caso Particular: Puntos Consecutivos Si los puntos son $x_i = i$, precalculamos prefijos y sufijos de $pref_i = \prod_{j=0}^{i-1}(X - j)$ y $suf_i =$

$\prod_{j=i+1}^{n}(X-j)$:

$$P_n(X) = \sum_{i=0}^{n} y_i \cdot \frac{pref_i \cdot suf_i}{(-1)^{n-i}\, i!\, (n-i)!}$$

T: $O(N)$ | **M:** $O(N)$ | *Para una única consulta* $P_n(x)$

► **Aritmética**

• ***Sumas de Gauss***

Para $k \geq 1$, sea $\Sigma_k(n) = \sum_{i=1}^{n} i^k$, con $\Sigma_0(n) = n$.

$$\Sigma_1(n) = \frac{n(n+1)}{2}, \quad \Sigma_2(n) = \frac{n(n+1)(2n+1)}{6}$$

$$\Sigma_3(n) = (\Sigma_1(n))^2\,, \quad \Sigma_4(n) = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

Usando recurrencia:

$$\Sigma_k(n) = \frac{(n+1)^{k+1} - 1 - \sum_{j=0}^{k-1} \binom{k+1}{j} \Sigma_j(n)}{k+1}.$$

T: $O(k^2)$ | **M:** $O(1)$ |

Usando interpolación de lagrange: Se evalúa la función de suma en los primeros $k+2$ puntos consecutivos $0, 1, \ldots, k+1$ y se interpola el valor de n utilizando Lagrange pero solo respone para una consulta, si se quieren varias consultas, es mejor usar los métodos siguientes.

T: $O(k)$ | **M:** $O(1)$ |

Usando Stirling:(Multiples consultas)

$$\Sigma_k(n) = \sum_{j=0}^{k} \left\{ {k \atop j} \right\} j! \binom{n+1}{j+1}$$

T: $O(k^2)$ | **M:** $O(k)$ | *Nums de Stirling por recurrencia. Eval* $O(k)$
T: $O(k \log k)$ | **M:** $O(k)$ | *Numeros de Stirling por Multiplicacion de Polinomios (NTT). Evaluar en* $O(k)$

Usando Bernoulli Numbers:(Multiples consultas)

$$\sum_{i=1}^{n} i^k = \frac{1}{k+1} \sum_{i=0}^{k} \binom{k+1}{i} B_i \cdot (n+1)^{k+1-i}$$

$$= k! \sum_{i=0}^{k} \frac{B_i \cdot (n+1)^{k+1-i}}{(k+1-i)!\, i!}$$

T: $O(k^2)$ | **M:** $O(k)$ | B_i *por recurrencia. Eval en* $O(k)$
T: $O(k \log k)$ | **M:** $O(k)$ | B_i *por NTT e inversión de series de potencias. Evaluar en* $O(k)$

• ***Numeros simplices (Sumas de sumas de Gauss)***

$$S_0(n) = 1, \quad S_1(n) = \sum_{i=1}^{n} 1 = \binom{n}{1}, \quad S_2(n) = \sum_{i=1}^{n} i = \binom{n+1}{2}$$

$$S_3(n) = \sum_{i=1}^{n} \left(\sum_{j=1}^{i} j \right) = \sum_{i=1}^{n} S_2(i) = \binom{n+2}{3},$$

$$S_k(n) = \sum_{i=1}^{n} S_{k-1}(i) = \binom{n+k-1}{k}.$$

- $S_k(n)$ es un polinomio de grado k en n. Necesitas $k+1$ puntos para interpolar.
- $S_k(0) = 0$ para $k \geq 1$ (suma vacía).
- $\binom{n+k-1}{k}$ es el número de multiconjuntos de tamaño k tomados de n elementos (estrellas y barras).

• ***Potencias de base fija (geométrica)***

Para $p \neq 1$:

$$\sum_{i=a}^{b} p^i = \frac{p^{b+1} - p^a}{1-p}, \quad \sum_{i=1}^{n} i\, p^i = \frac{p - (n+1)p^{n+1} + np^{n+2}}{(1-p)^2}.$$

$$\sum_{i=0}^{n} p^i = \frac{1-p^{n+1}}{1-p}, \quad \sum_{i=1}^{n} p^i = \frac{p(1-p^n)}{1-p},$$

Si $p = 1$, las cuatro sumas son, respectivamente, $b-a+1$, $n(n+1)/2$, $n+1$ y n.

• ***Series infinitas***

Las siguientes fórmulas requieren $|p| < 1$:

$$\sum_{i=0}^{\infty} p^i = \frac{1}{1-p}, \qquad \sum_{i=1}^{\infty} p^i = \frac{p}{1-p},$$

$$\sum_{i=1}^{\infty} i\, p^i = \frac{p}{(1-p)^2}, \qquad \sum_{i=1}^{\infty} i^2 p^i = \frac{p(1+p)}{(1-p)^3}.$$

• ***Diferencias finitas***

Si a_t es polinómica de grado $\leq k$, entonces $\Delta^{k+1} a_t = 0$. Definimos

$$\Delta^0 a_t = a_t, \qquad \Delta^{j+1} a_t = \Delta^j a_{t+1} - \Delta^j a_t.$$

Con los primeros $k+1$ términos obtenemos:

$$a_n = \sum_{j=0}^{k} \binom{n-1}{j} \Delta^j a_1, \qquad \sum_{t=1}^{n} a_t = \sum_{j=0}^{k} \binom{n}{j+1} \Delta^j a_1.$$

T: $O(k^2)$ | **M:** $O(k)$ | *Construir las diferencias desde* $k+1$ *términos; evaluar en* $O(k)$

► **Geometría**

Polígonos: área y puntos enteros Vértices $P_i = (x_i, y_i)$ en orden; índices módulo n. **T:** $O(n)$ | **M:** $O(1)$ |

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} P_i \times P_{i+1} \right|.$$

Para un polígono simple sin agujeros y con vértices enteros:

$$B = \sum_{i=0}^{n-1} \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|),$$

$$A = I + \frac{B}{2} - 1, \qquad I = \frac{2A - B + 2}{2}.$$

B: puntos enteros del borde; I: puntos enteros interiores. Un segmento entero no degenerado contiene $\gcd(|\Delta x|, |\Delta y|) + 1$ puntos enteros.

Triángulos: Herón, radios y trigonometría Lados a, b, c opuestos a α, β, γ; $s = (a+b+c)/2$, área $A > 0$, inradio r, circunradio R.

$$A = \sqrt{s(s-a)(s-b)(s-c)}, \qquad r = \frac{A}{s}, \qquad R = \frac{abc}{4A}.$$

$$a^2 = b^2 + c^2 - 2bc\cos\alpha, \qquad A = \frac{bc\sin\alpha}{2},$$

$$\frac{a}{\sin\alpha} = \frac{b}{\sin\beta} = \frac{c}{\sin\gamma} = 2R.$$

Cuadriláteros: Brahmagupta y Bretschneider Cuadrilátero convexo; lados a, b, c, d en orden, diagonales p, q, $s = (a+b+c+d)/2$. Sean α, γ ángulos opuestos y $T = (s-a)(s-b)(s-c)(s-d)$.

$$A^2 = T - abcd\cos^2\left(\frac{\alpha+\gamma}{2}\right) = T - \frac{(ac+bd)^2 - p^2q^2}{4}.$$

Si es cíclico:

$$A = \sqrt{T}, \qquad pq = ac + bd \quad \text{(Ptolomeo)}.$$

Círculos: intersecciones Centros O_1, O_2, radios $r_1, r_2 > 0$; $d = \|O_2 - O_1\| > 0$. Hay intersección de circunferencias si $|r_1 - r_2| \leq d \leq r_1 + r_2$.

$$e = \frac{O_2 - O_1}{d}, \qquad t = \frac{d^2 + r_1^2 - r_2^2}{2d}, \qquad h = \sqrt{r_1^2 - t^2},$$

$$X_\pm = O_1 + te \pm he^\perp.$$

$h = 0$: tangencia. Si $d = 0$, coinciden si $r_1 = r_2$; en otro caso no se intersectan.

Círculos tangentes: Descartes Cuatro círculos mutuamente tangentes; curvaturas $k_i = 1/r_i$, negativas para el círculo que encierra a los otros:

$$(k_1 + k_2 + k_3 + k_4)^2 = 2\left(k_1^2 + k_2^2 + k_3^2 + k_4^2\right)$$

Volumen de un simplex Vértices $P_0, \ldots, P_n \in \mathbb{R}^d$, $d \geq n$. Sea E la matriz con filas $P_i - P_0$, $i = 1, \ldots, n$:

$$V_n = \frac{1}{n!} \sqrt{\det\left(AA^\top\right)}.$$

Si $d = n$, se simplifica a $V_n = |\det A|/n!$.

► **Desigualdades**

AM-GM-HM: Igualdad ssi $x_1 = \cdots = x_n$

$$\frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}} \leq \sqrt[n]{\prod_{i=1}^{n} x_i} \leq \frac{1}{n} \sum_{i=1}^{n} x_i \quad (x_i > 0)$$

Media Pot: $M_r(x) = \left(\frac{1}{n}\sum x_i^r\right)^{1/r}$: Si $r < s$, ent, $M_r(x) \leq M_s(x)$

$M_{-1} = \text{HM} \quad \leq \quad M_0 = \text{GM} \quad \leq \quad M_1 = \text{AM} \quad \leq \quad M_2 = \text{RMS}$

Valor Absoluto / RMS: Válida para cualquier $x_i \in \mathbb{R}$:

$$\left| \frac{1}{n} \sum_{i=1}^{n} x_i \right| \leq \frac{1}{n} \sum_{i=1}^{n} |x_i| \leq \sqrt{\frac{1}{n} \sum_{i=1}^{n} x_i^2} = \text{RMS}$$

Cauchy-Schwarz: Igualdad ssi $\exists k \in \mathbb{R} : a_i = kb_i$

$$\left(\sum_{i=1}^{n} a_i b_i\right)^2 \leq \left(\sum_{i=1}^{n} a_i^2\right)\left(\sum_{i=1}^{n} b_i^2\right)$$

Forma de Engel (Sedrakyan): $b_i > 0$

$$\sum_{i=1}^{n} \frac{a_i^2}{b_i} \geq \frac{\left(\sum_{i=1}^{n} a_i\right)^2}{\sum_{i=1}^{n} b_i}$$

Hölder: Con $\frac{1}{p} + \frac{1}{q} = 1$ $(p, q > 1)$.

$$\sum |a_i b_i| \leq \left(\sum |a_i|^p\right)^{1/p} \left(\sum |b_i|^q\right)^{1/q}$$

Minkowski: $p \geq 1$

$$\left(\sum |a_i + b_i|^p\right)^{1/p} \leq \left(\sum |a_i|^p\right)^{1/p} + \left(\sum |b_i|^p\right)^{1/p}$$

Reagrupamiento: Para $a_1 \leq \cdots \leq a_n$ y $b_1 \leq \cdots \leq b_n$:

$$\sum_{i=1}^{n} a_i b_{n-i+1} \leq \sum_{i=1}^{n} a_i b_{\sigma(i)} \leq \sum_{i=1}^{n} a_i b_i$$

Chebyshev: Si $a_1 \leq \cdots \leq a_n$ y $b_1 \leq \cdots \leq b_n$ (Orden opuesto, desigualdad se invierte.)

$$\frac{1}{n} \sum a_i b_i \geq \left(\frac{1}{n} \sum a_i\right)\left(\frac{1}{n} \sum b_i\right)$$

Jensen: Para f convexa, $w_i \geq 0$ y $\sum w_i = 1$

$$f\left(\sum w_i x_i\right) \leq \sum w_i f(x_i)$$

Bernoulli: Para $x \geq -1$, con $r \geq 1$ o $r \leq 0$. Si $0 \leq r \leq 1$, desigualdad se invierte

$$(1 + x)^r \geq 1 + rx$$

Karamata: Si $A \succ B$ (A mayoriza a B) y f es convexa

$$\sum_{i=1}^{n} f(a_i) \geq \sum_{i=1}^{n} f(b_i)$$

Schur: Para $x, y, z \geq 0,\ r > 0$:

$$x^r(x-y)(x-z) + y^r(y-z)(y-x) + z^r(z-x)(z-y) \geq 0$$